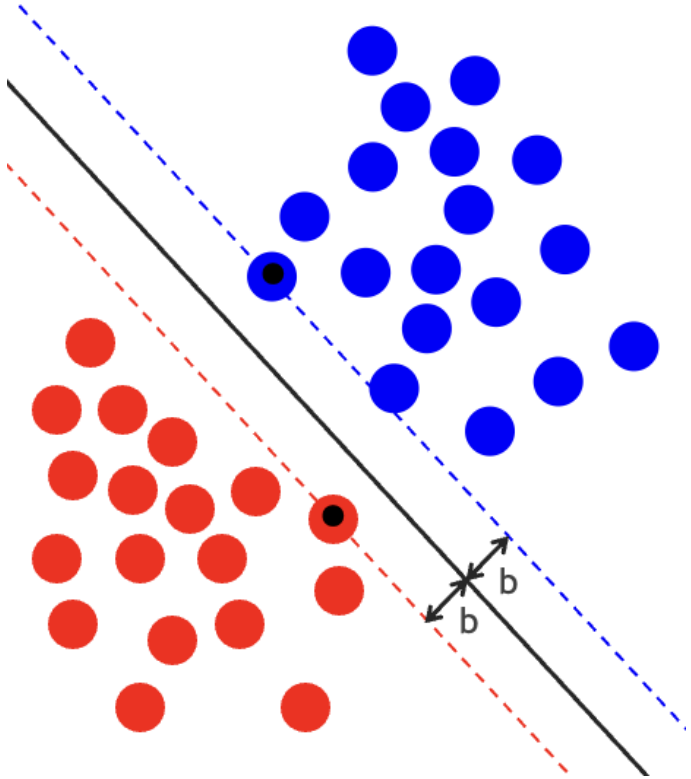
Reconocimiento de Patrones

Versión 2026



SVM: Support Vector Machines

(CAPÍTULO 4 – PR.04.C)



Domingo Mery

Ciencia de la Computación
Universidad Católica – Chile
http://domingomery.ing.uc.cl

Historia



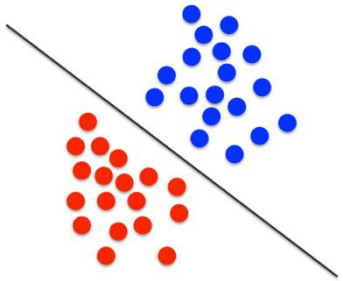
Chervonenkis & Vapnik

El algoritmo Support Vector Machine fue desarrollado por dos matemáticos rusos, Vladimir Vapnik y Alexey Chervonenkis, en 1963. Vapnik siguió desarrollando el algoritmo a lo largo de su carrera y, en 1992, creó una versión no lineal del clasificador utilizando el ”kernel-trick" mientras trabajaba con Bernhard Boser e Isabelle Guyon. En 1993 propuso con Corinna Cortes la versión no lineal de margen suave, que se lanzó en 1995.

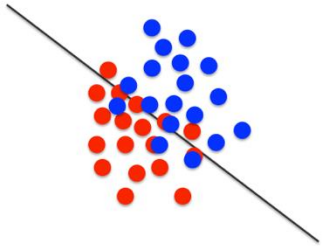
https://medium.com/@nickdarby/data-chef-episode-8-svms-7fb495e96d90
https://en.wikipedia.org/wiki/Support_vector_machine#History

SVM: Máquinas vectoriales de soporte

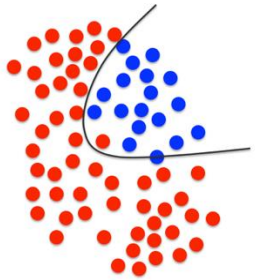
1) Lineal con separación perfecta



2) Lineal sin separación perfecta

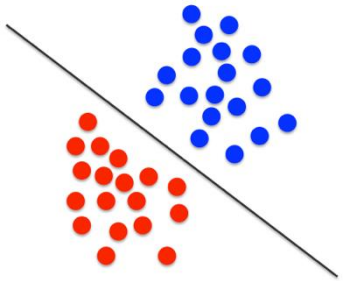


3) No lineal



SVM: Máquinas vectoriales de soporte

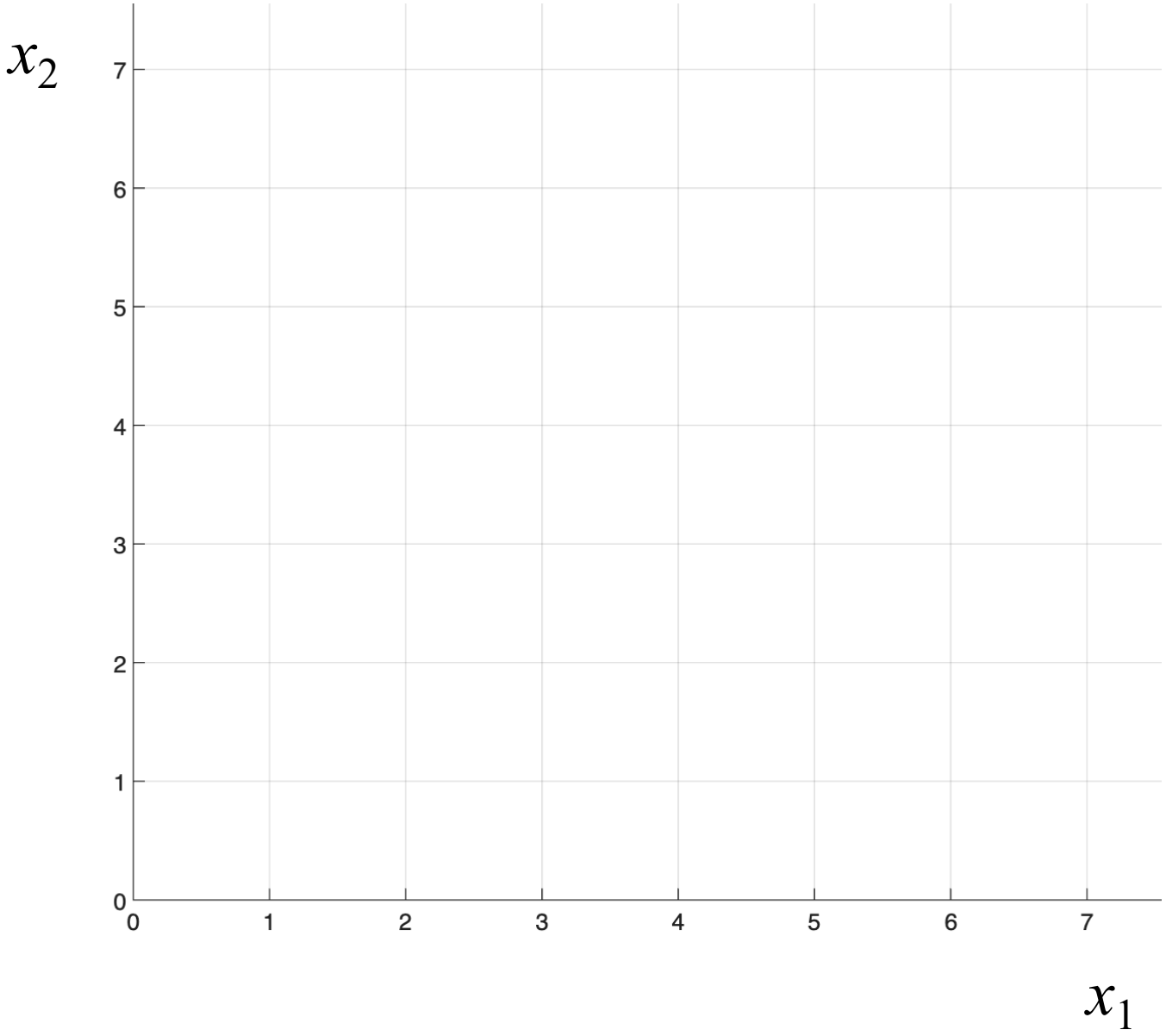
1) Lineal con separación perfecta



2) Lineal sin separación perfecta

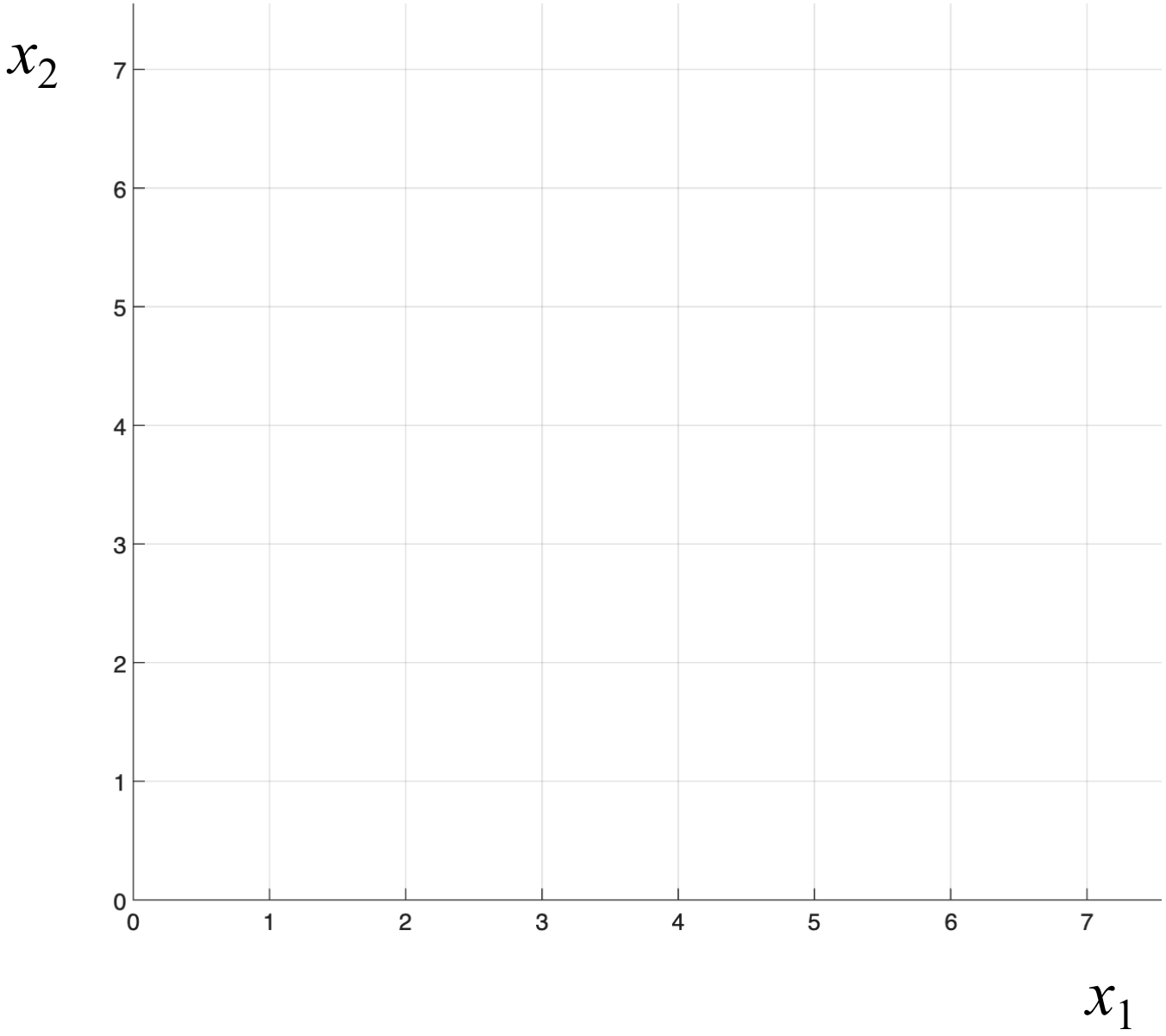
3) No lineal

Ejemplo:
dos clases y
distancia mínima



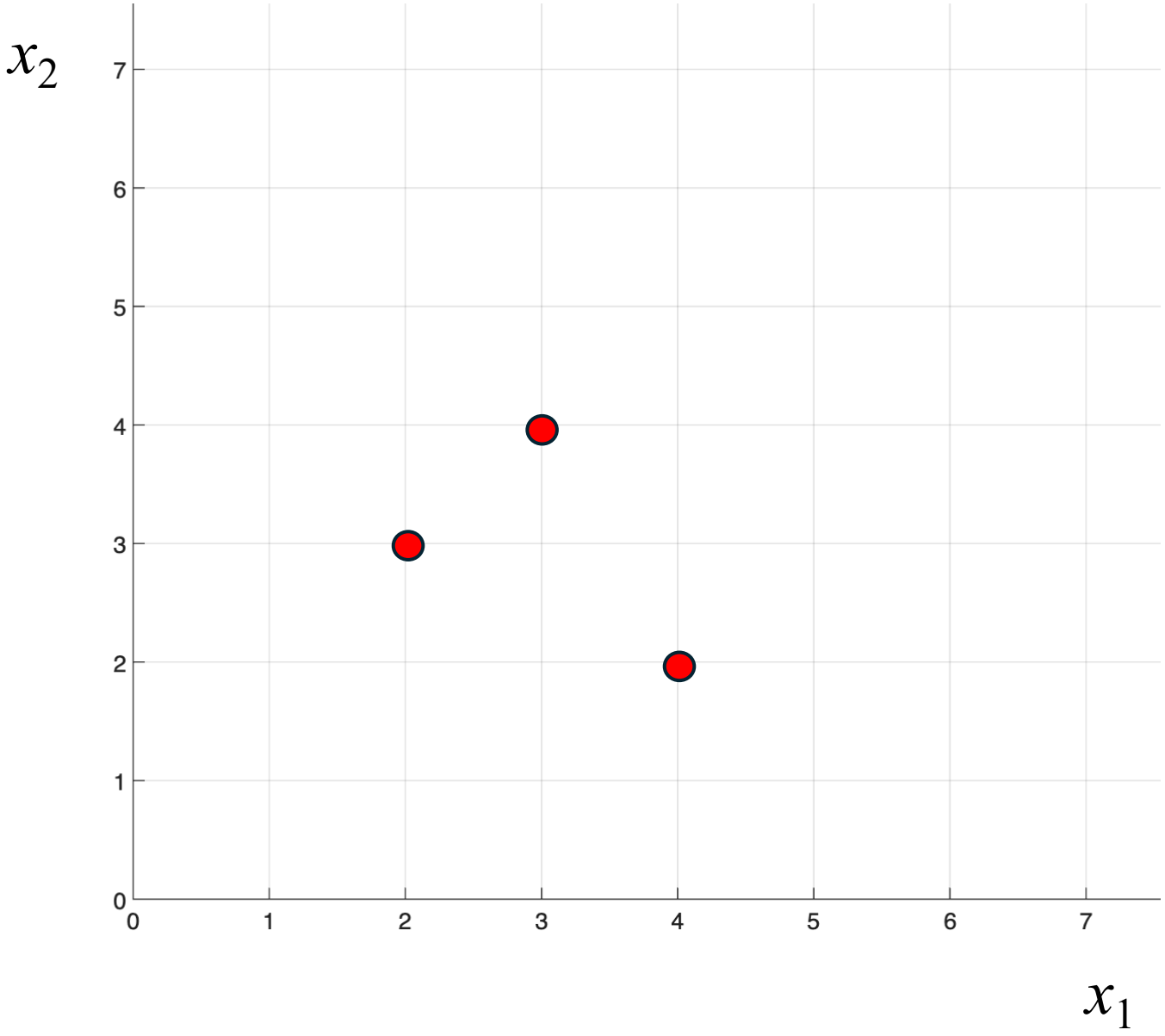
Ejemplo:
dos clases y
distancia mínima

#	x_1	x_2	y
1	4	2	0
2	3	4	0
3	2	3	0
4	6	4	1
5	5	6	1
6	4	5	1



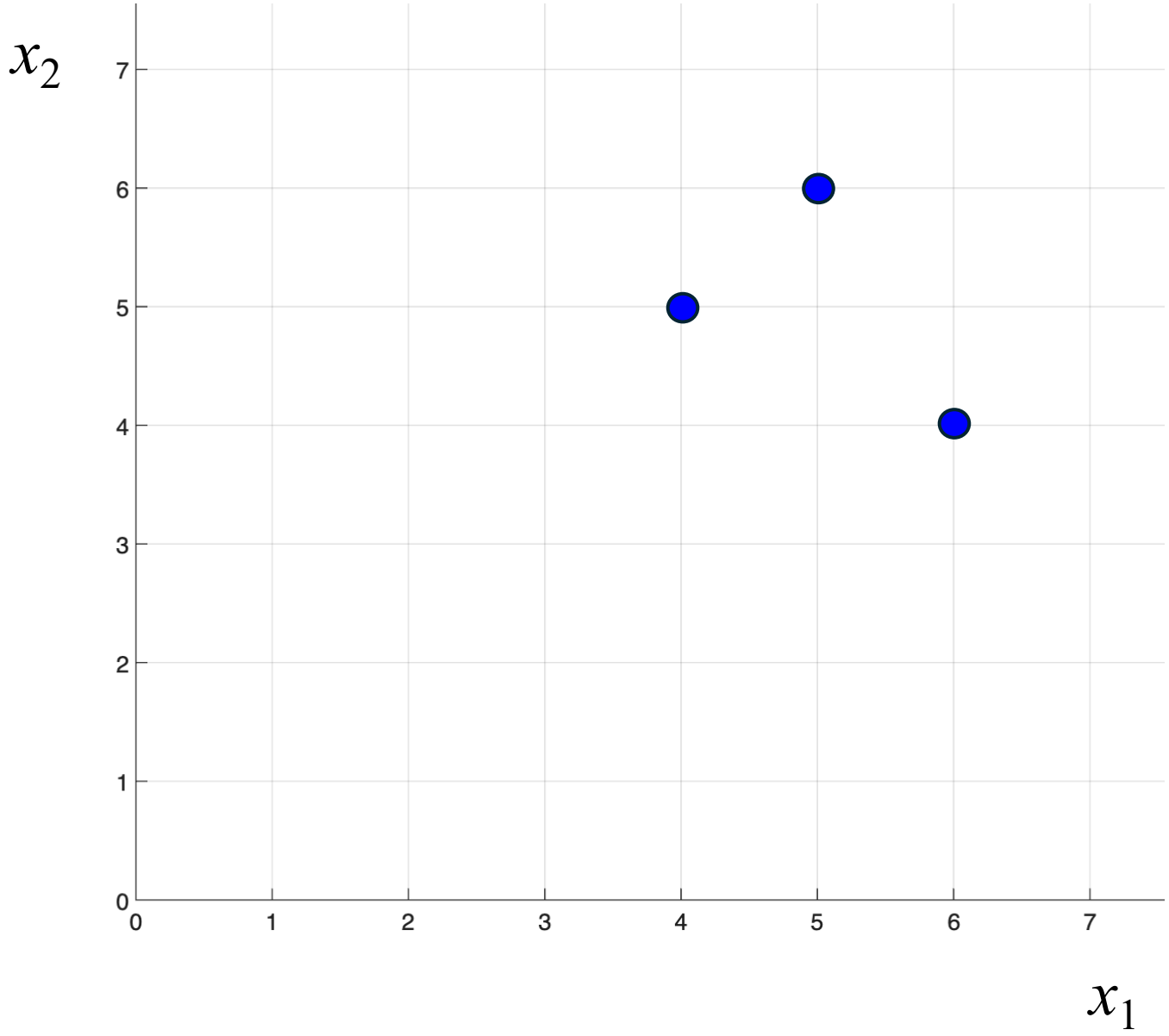
Ejemplo: dos clases y distancia mínima

#	x_1	x_2	y
1	4	2	0
2	3	4	0
3	2	3	0
4	6	4	1
5	5	6	1
6	4	5	1



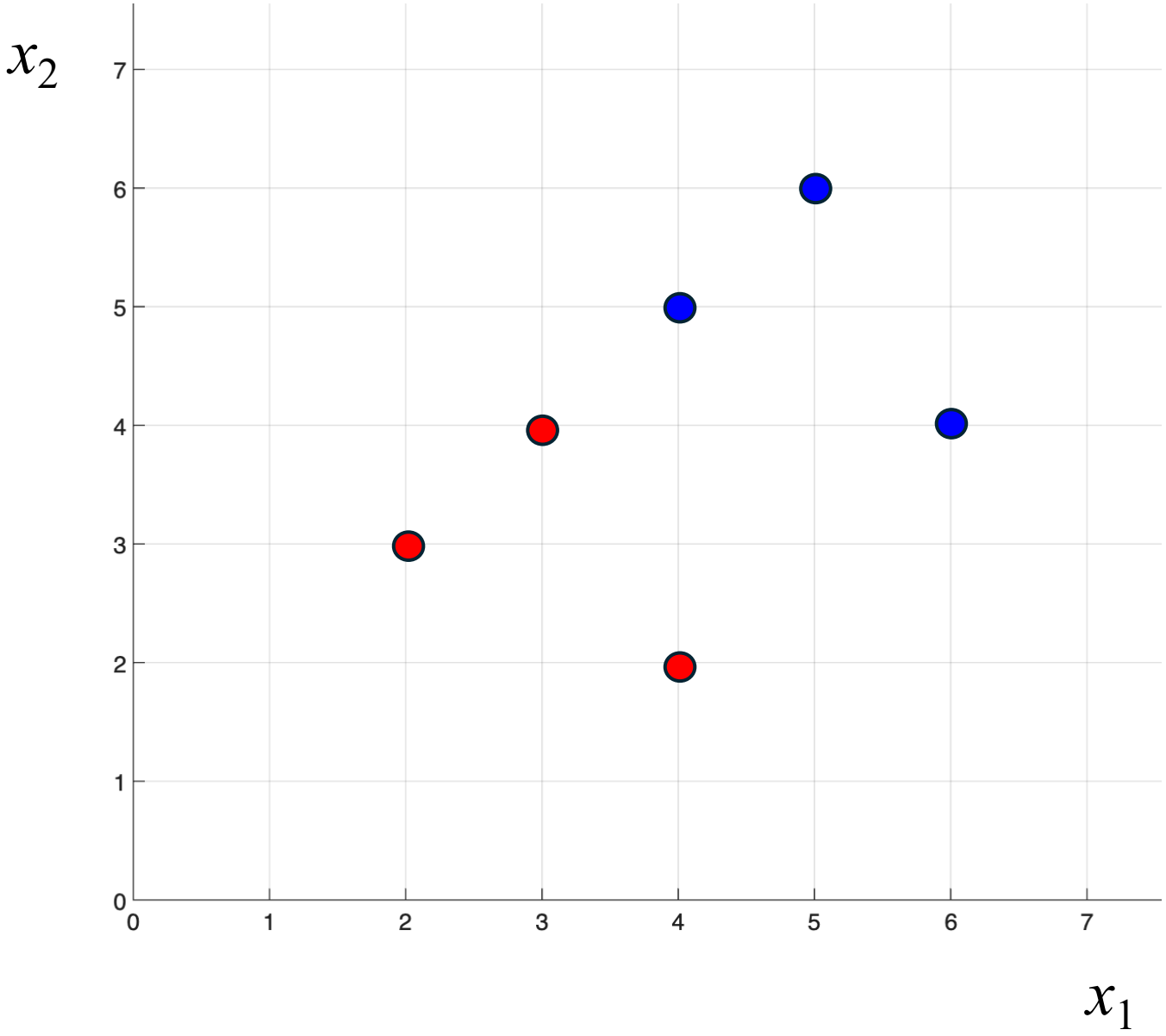
Ejemplo:
dos clases y
distancia mínima

#	x_1	x_2	y
1	4	2	0
2	3	4	0
3	2	3	0
4	6	4	1
5	5	6	1
6	4	5	1



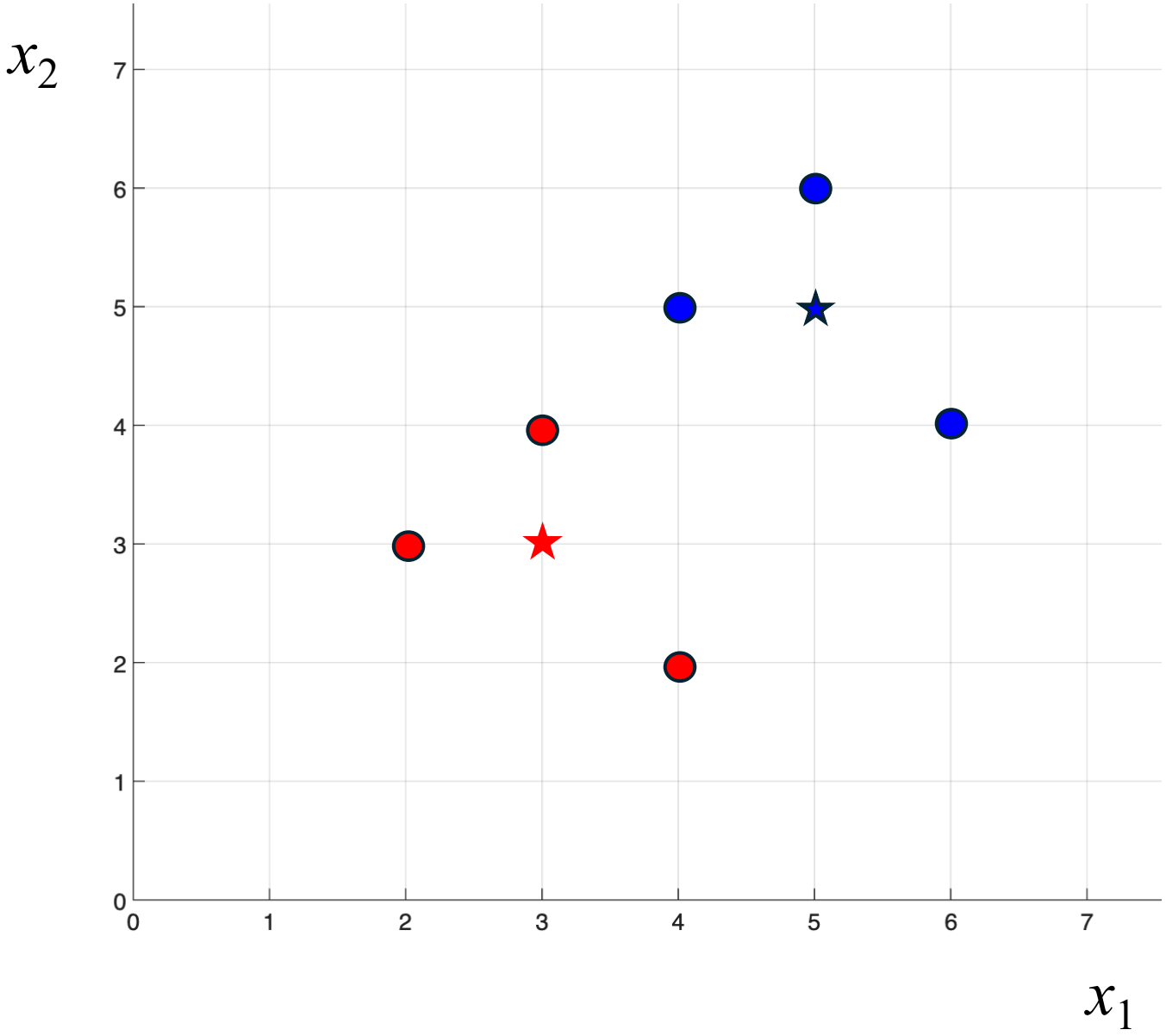
Ejemplo:
dos clases y
distancia mínima

#	x_1	x_2	y
1	4	2	0
2	3	4	0
3	2	3	0
4	6	4	1
5	5	6	1
6	4	5	1



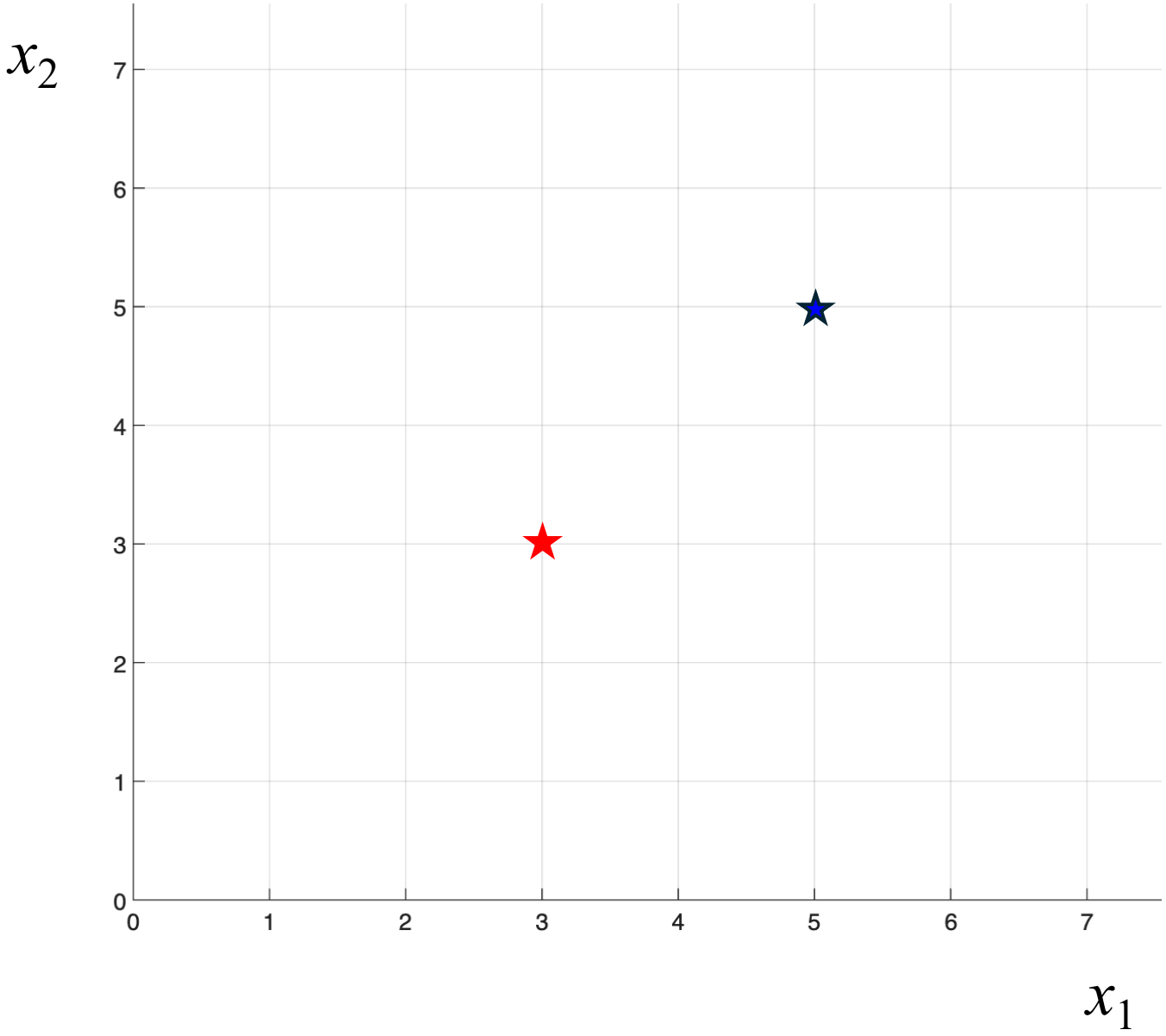
Ejemplo:
dos clases y
distancia mínima

#	x_1	x_2	y
1	4	2	0
2	3	4	0
3	2	3	0
4	6	4	1
5	5	6	1
6	4	5	1



Ejemplo: dos clases y distancia mínima

#	x_1	x_2	y
1	4	2	0
2	3	4	0
3	2	3	0
4	6	4	1
5	5	6	1
6	4	5	1

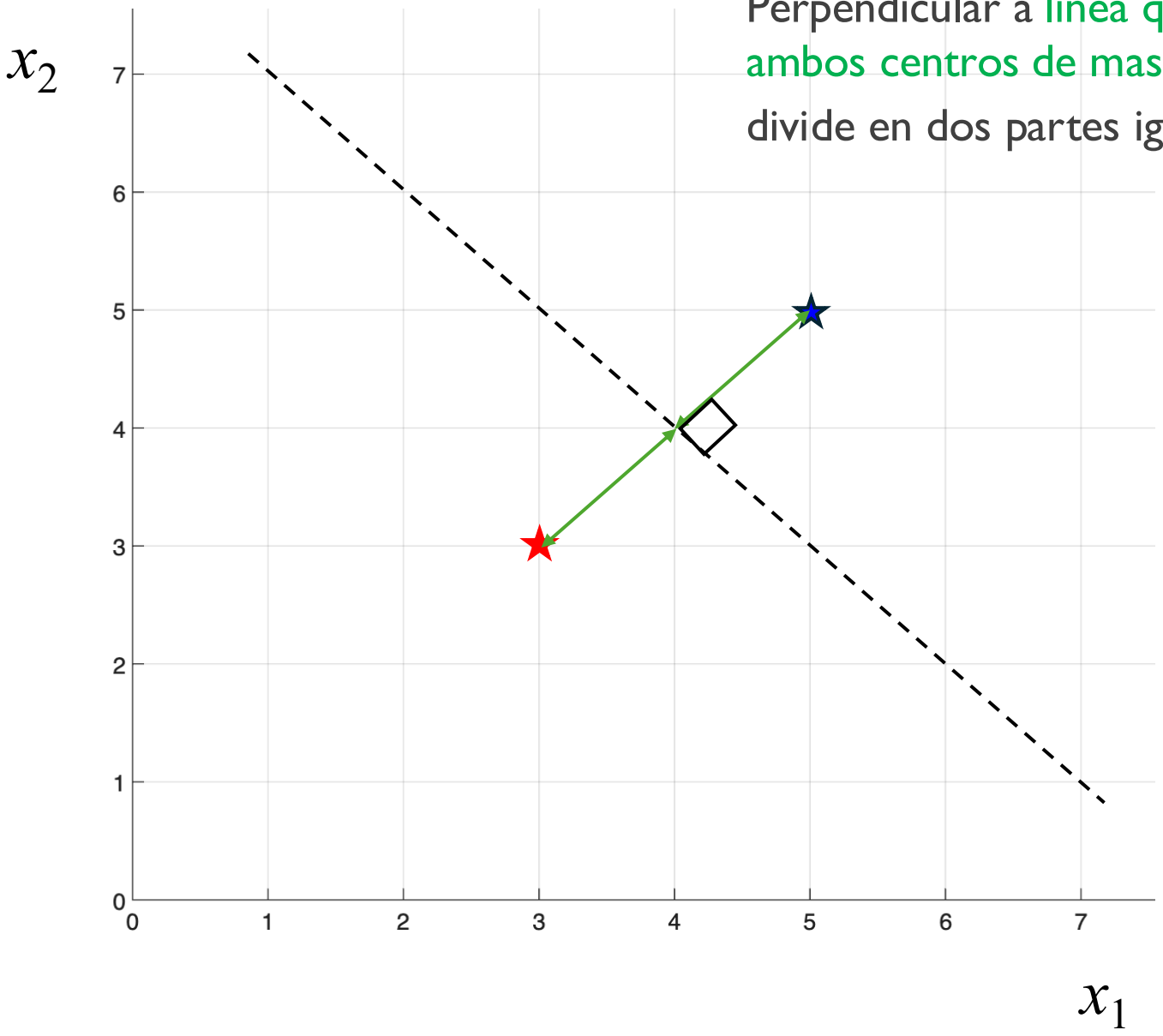


Ejemplo:
dos clases y distancia mínima

#	x_1	x_2	y
1	4	2	0
2	3	4	0
3	2	3	0
4	6	4	1
5	5	6	1
6	4	5	1

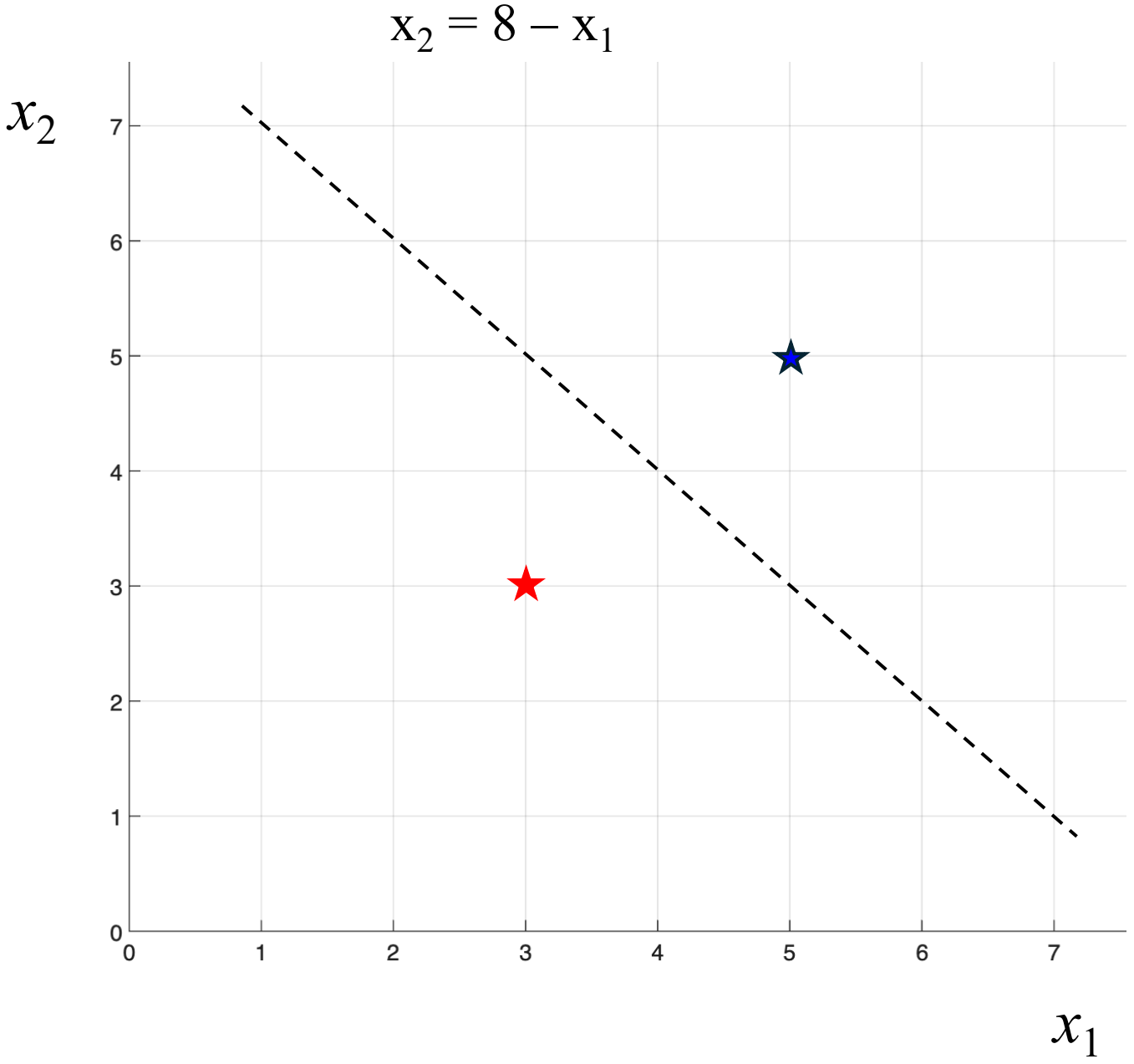
Línea de decisión:

Perpendicular a línea que une a ambos centros de masa, y que la divide en dos partes iguales



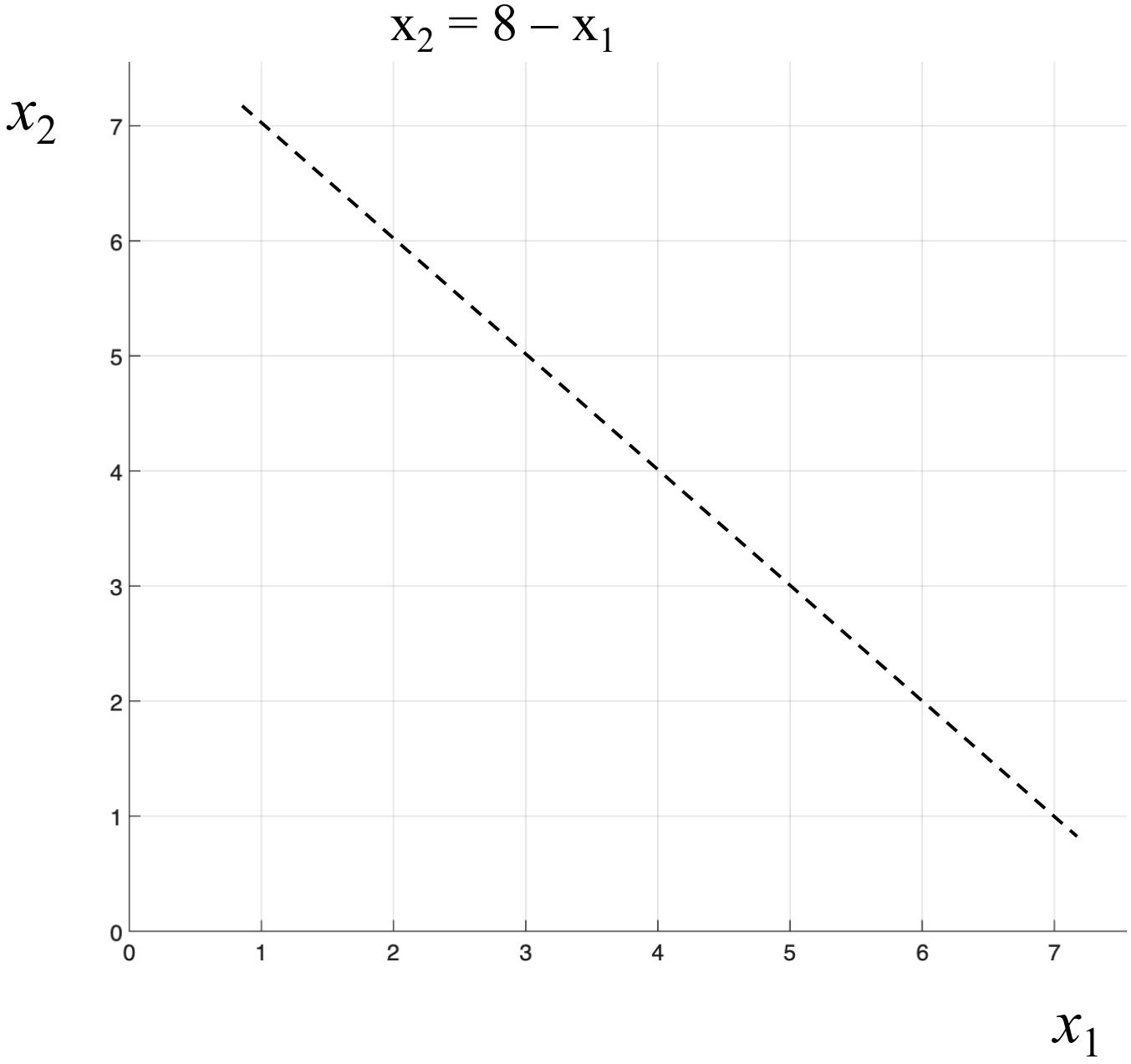
Ejemplo:
dos clases y
distancia mínima

#	x_1	x_2	y
1	4	2	0
2	3	4	0
3	2	3	0
4	6	4	1
5	5	6	1
6	4	5	1



Ejemplo:
dos clases y
distancia mínima

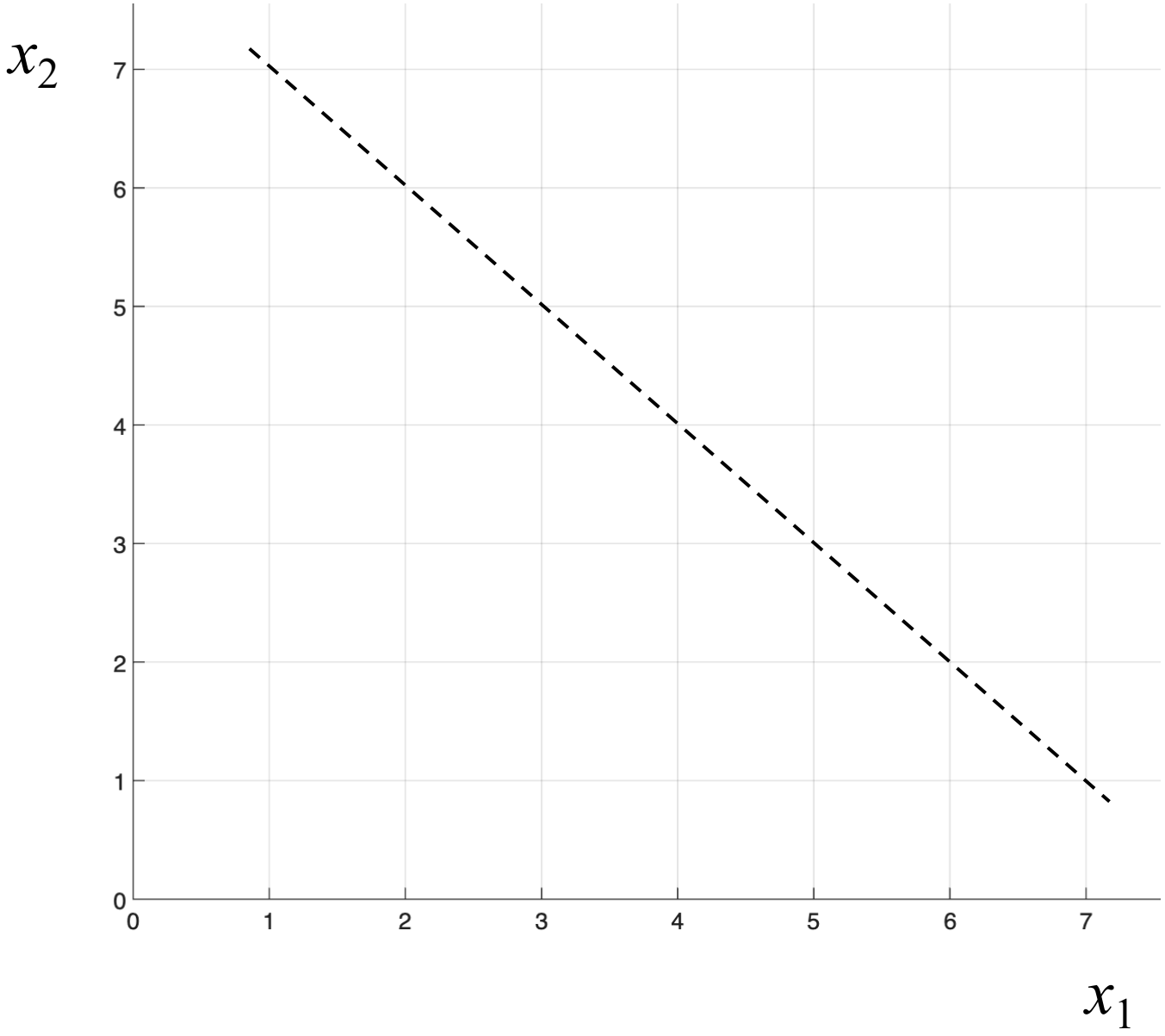
#	x_1	x_2	y
1	4	2	0
2	3	4	0
3	2	3	0
4	6	4	1
5	5	6	1
6	4	5	1



Ejemplo:
dos clases y
distancia mínima

#	x_1	x_2	y
1	4	2	0
2	3	4	0
3	2	3	0
4	6	4	1
5	5	6	1
6	4	5	1

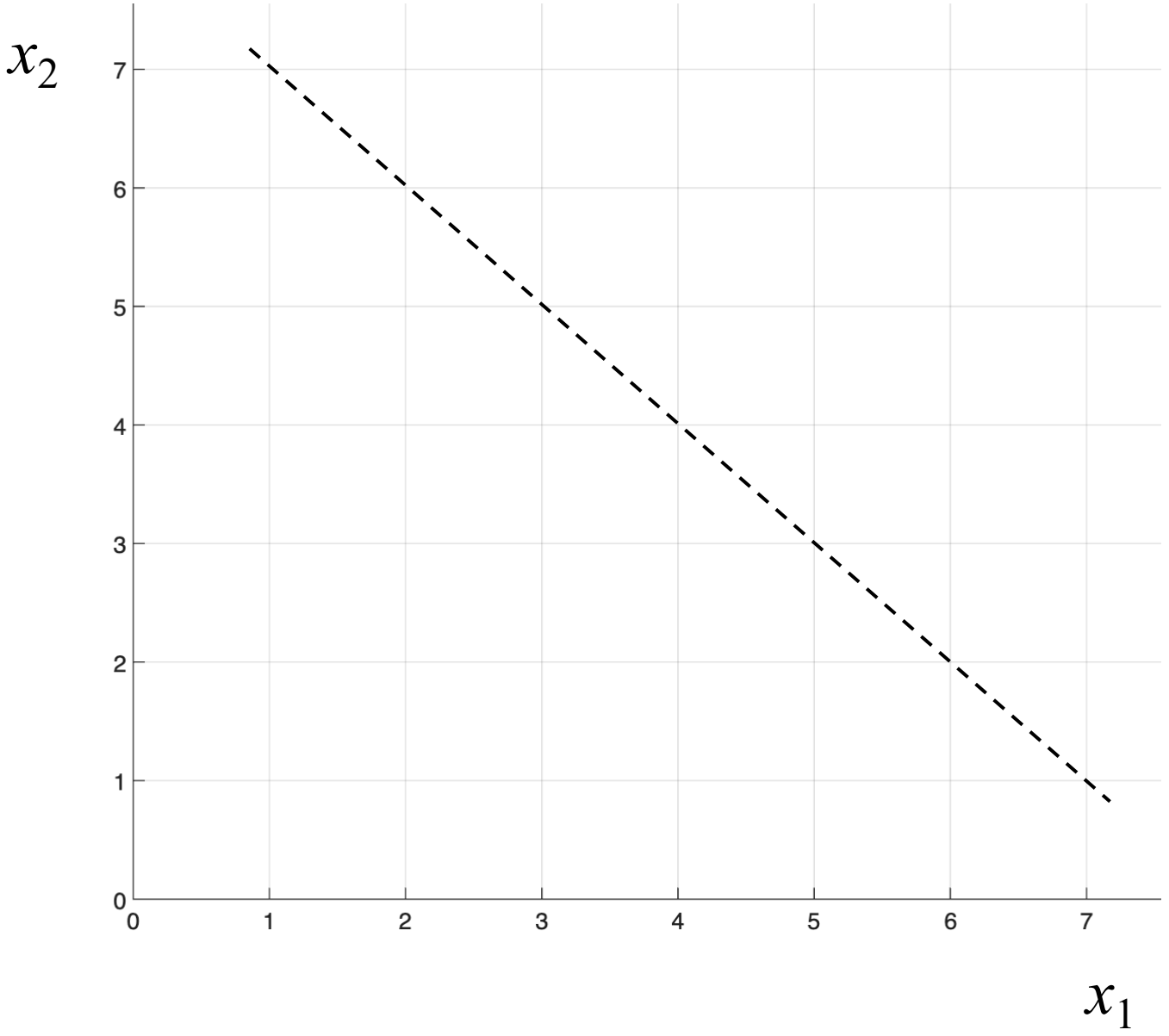
$$x_2 = 8 - x_1 \Rightarrow x_2 + x_1 - 8 = 0$$



Ejemplo:
dos clases y
distancia mínima

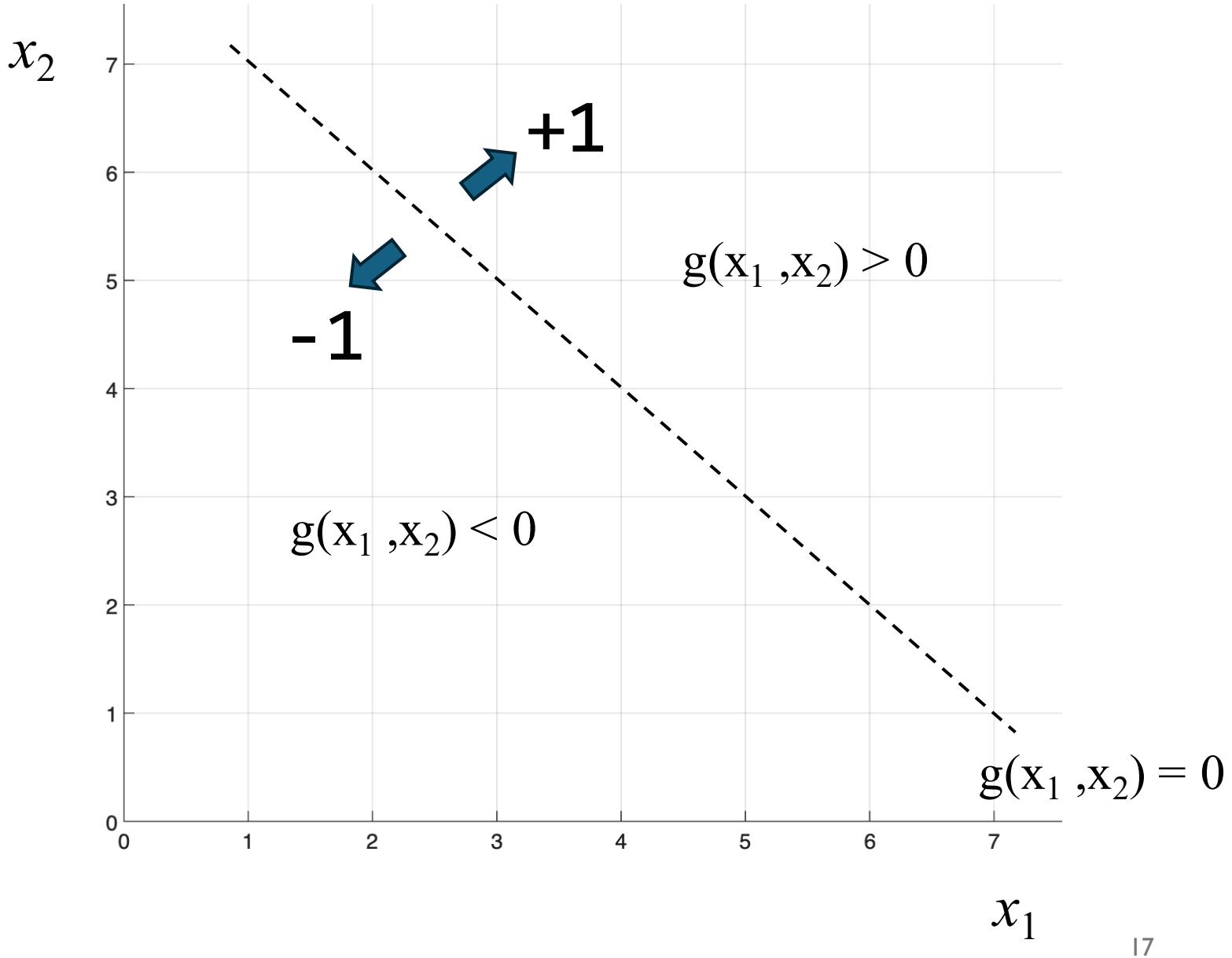
$$x_2 = 8 - x_1 \Rightarrow g(x_1, x_2) = x_2 + x_1 - 8 = 0$$

#	x_1	x_2	y
1	4	2	0
2	3	4	0
3	2	3	0
4	6	4	1
5	5	6	1
6	4	5	1



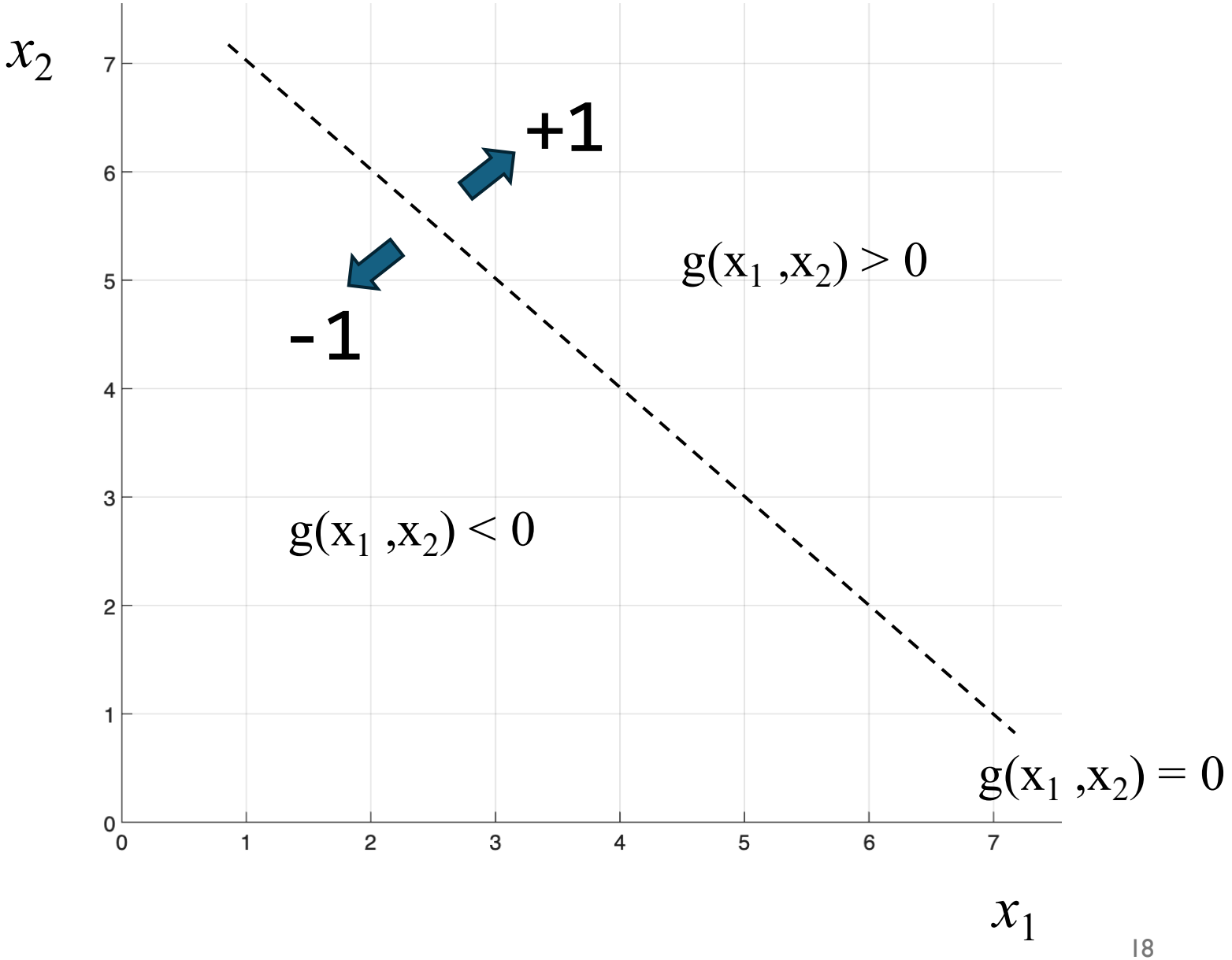
Ejemplo: dos clases y distancia mínima

#	x_1	x_2	y
1	4	2	0
2	3	4	0
3	2	3	0
4	6	4	1
5	5	6	1
6	4	5	1



Ejemplo: dos clases y distancia mínima

#	x_1	x_2	y
1	4	2	-1
2	3	4	-1
3	2	3	-1
4	6	4	+1
5	5	6	+1
6	4	5	+1



Ejemplo:
dos clases y
distancia mínima

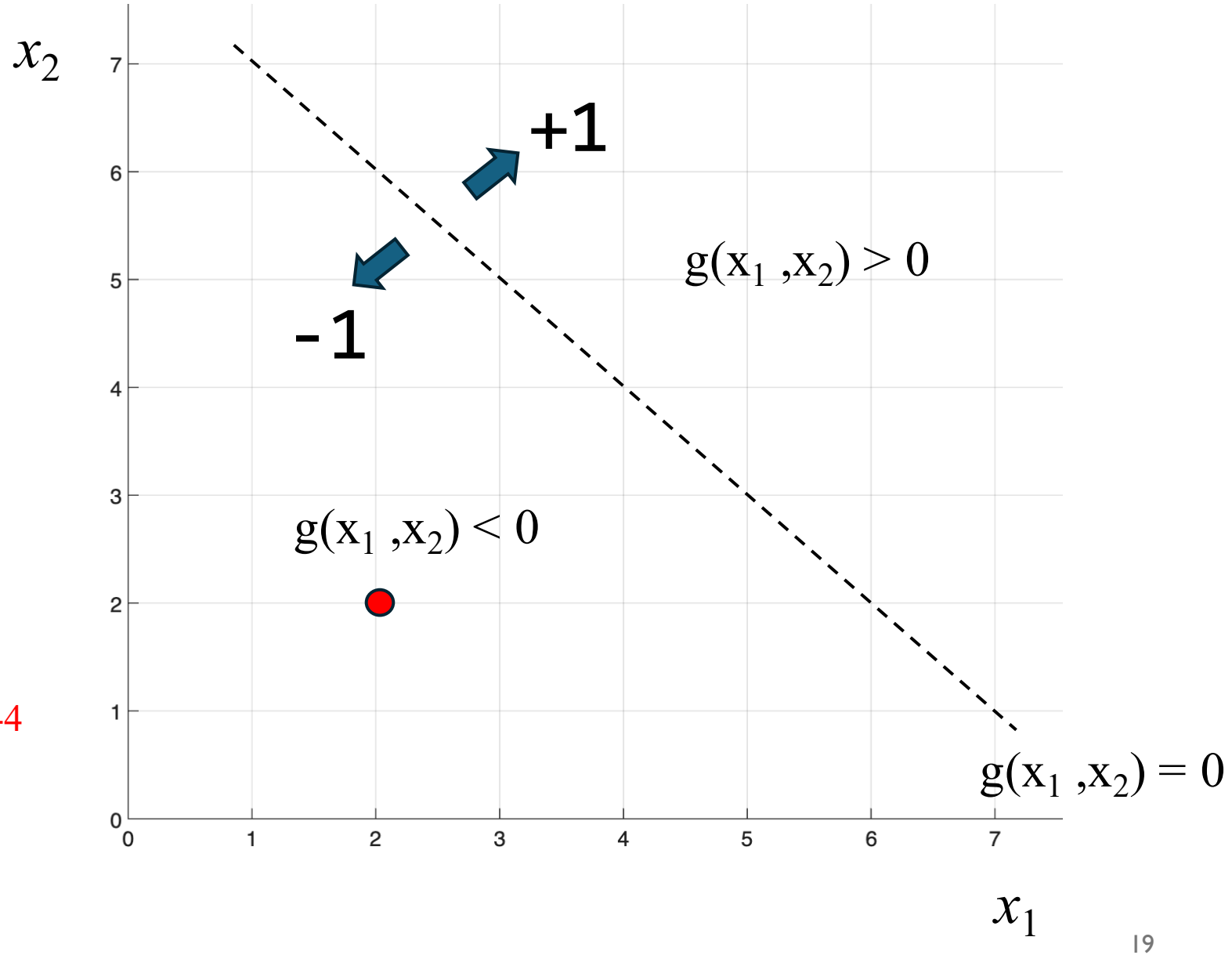
Algoritmo de Clasificación:

if $g(x_1, x_2) > 0$ then clase = +1
else clase = -1

$g(x_1, x_2) = x_2 + x_1 - 8$

Testing:

$(x_1,x_2)=(2,2)$: $g(x_1, x_2) = 2+2-8 = -4$ <0 entonce clase -1



Ejemplo:
dos clases y
distancia mínima

Algoritmo de Clasificación:

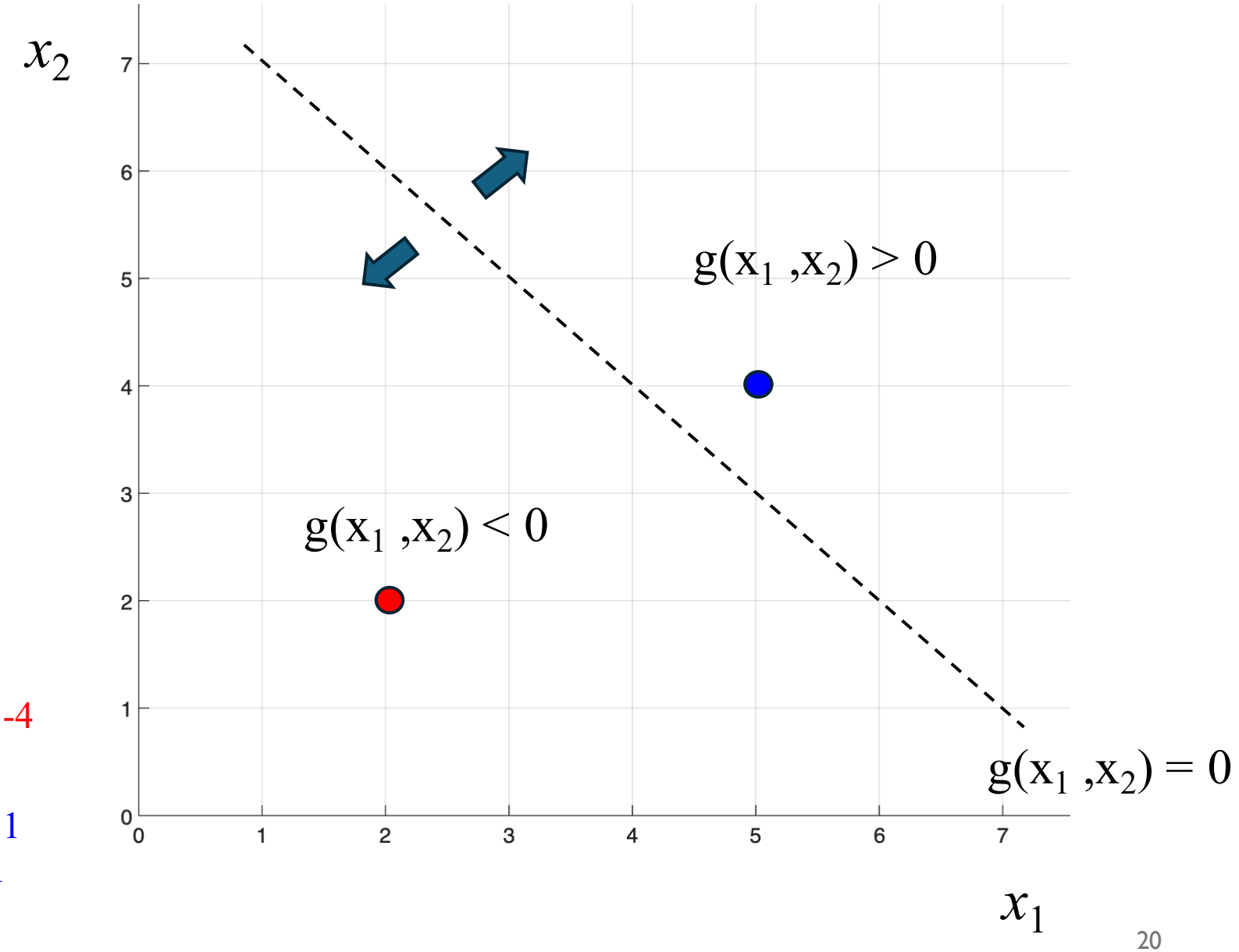
if $g(x_1, x_2) > 0$ then clase = +1
else clase = -1

$g(x_1, x_2) = x_2 + x_1 - 8$

Testing:

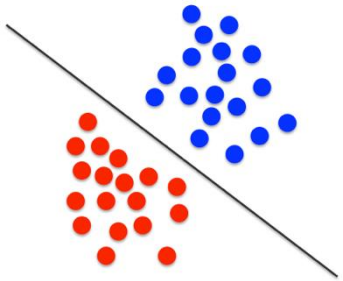
$(x_1,x_2)=(2,2)$: $g(x_1, x_2) = 2+2-8 = -4$
<0 entonce clase -1

$(x_1,x_2)=(5,5)$: $g(x_1, x_2) = 5+4-8 = 1$
<0 entonce clase +1



SVM: Máquinas vectoriales de soporte

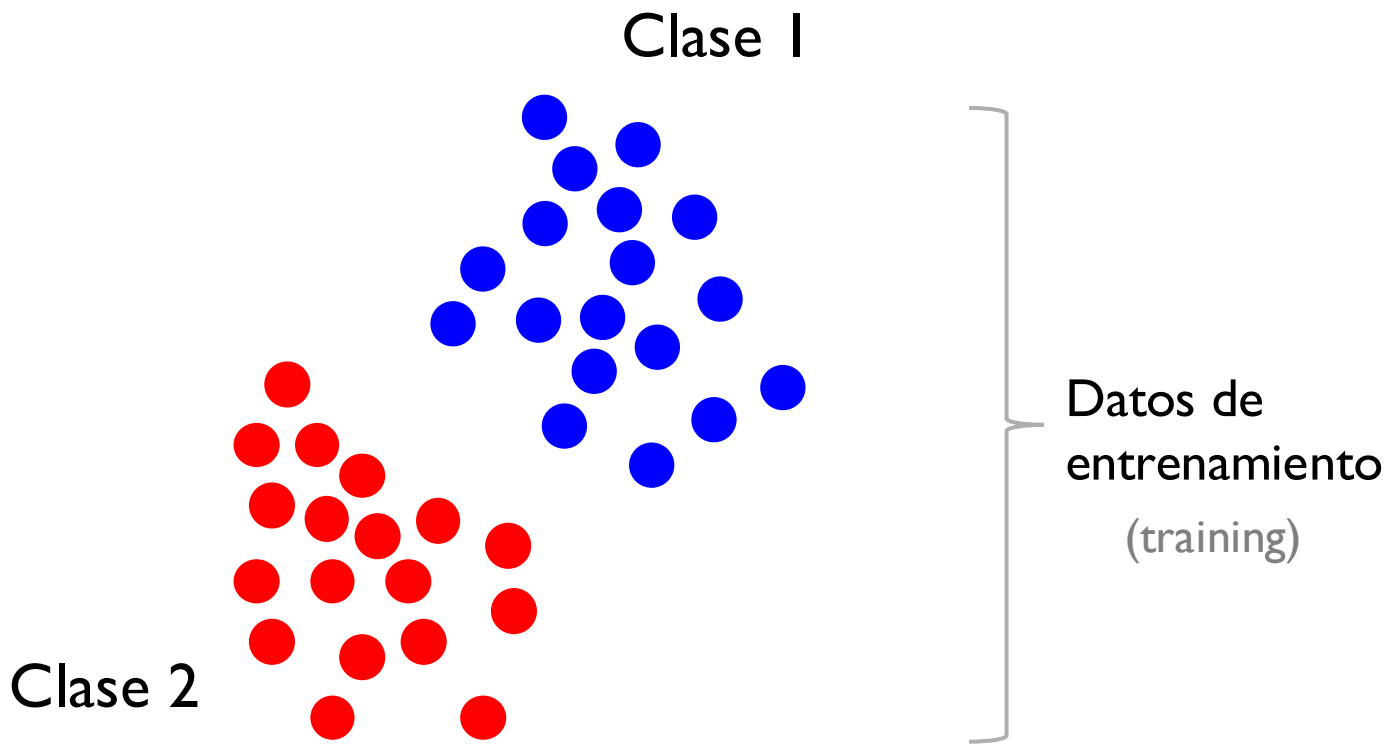
1) Lineal con separación perfecta



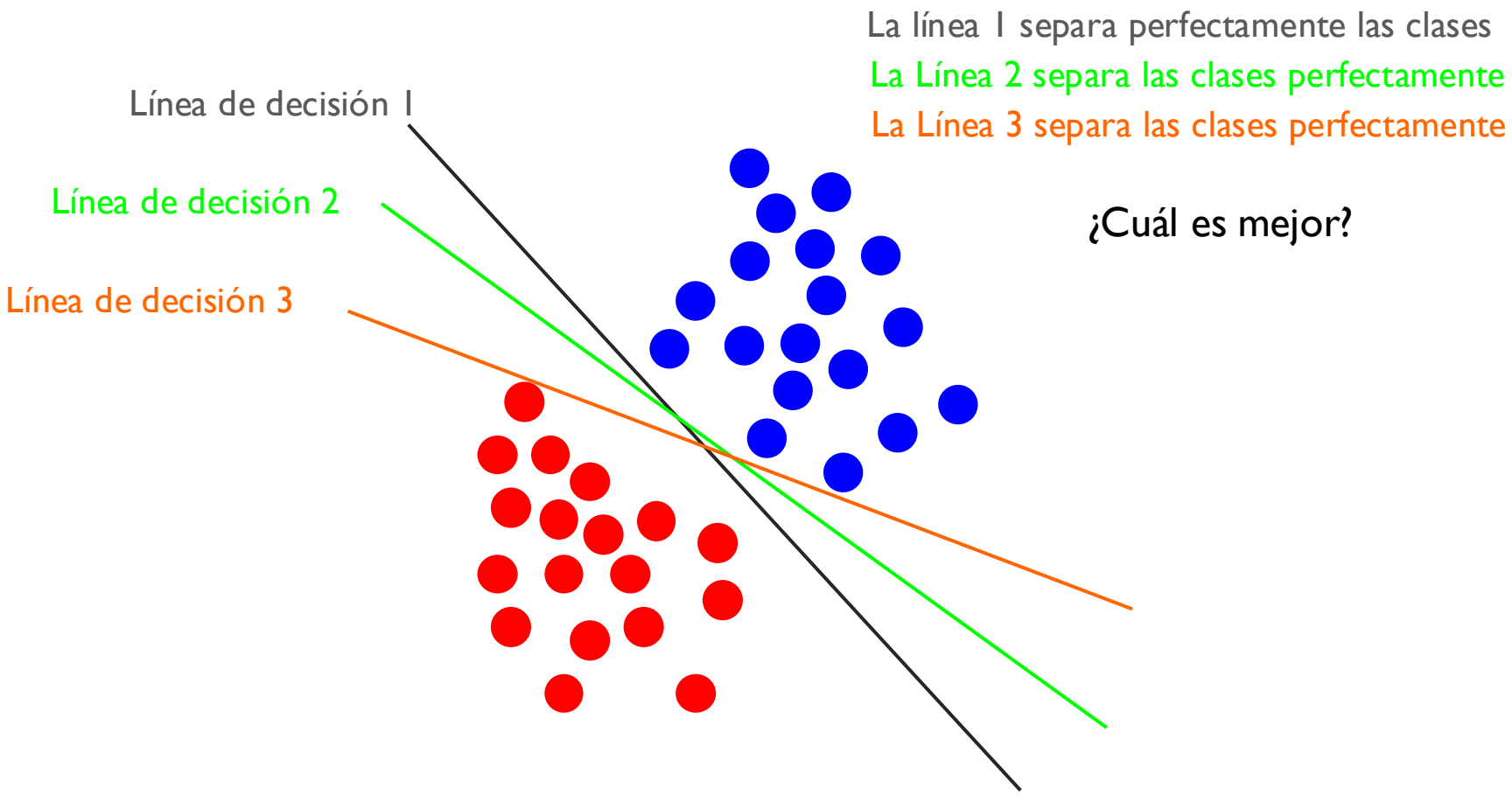
2) Lineal sin separación perfecta

3) No lineal

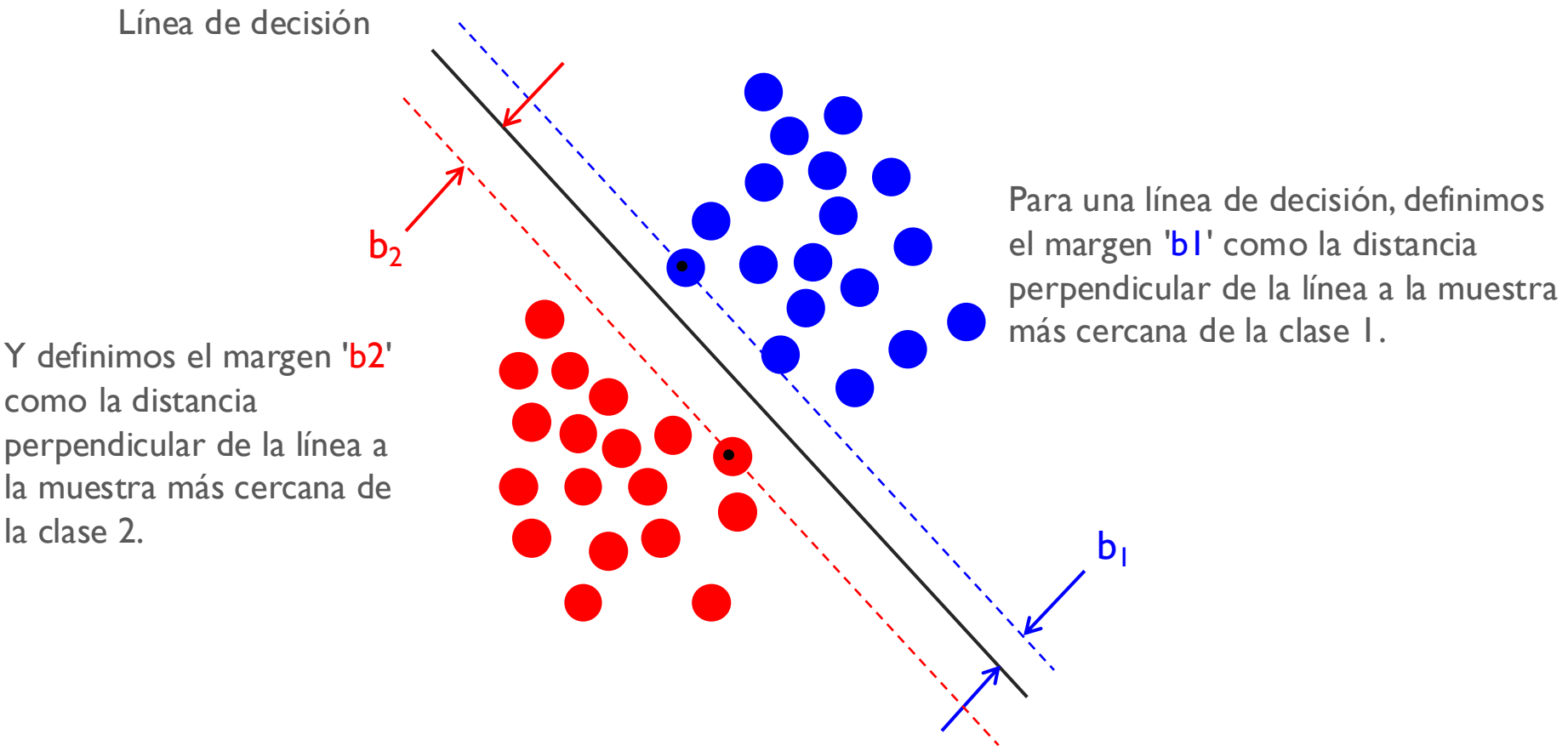
SVM: dos clases



SVM: dos clases



SVM: dos clases

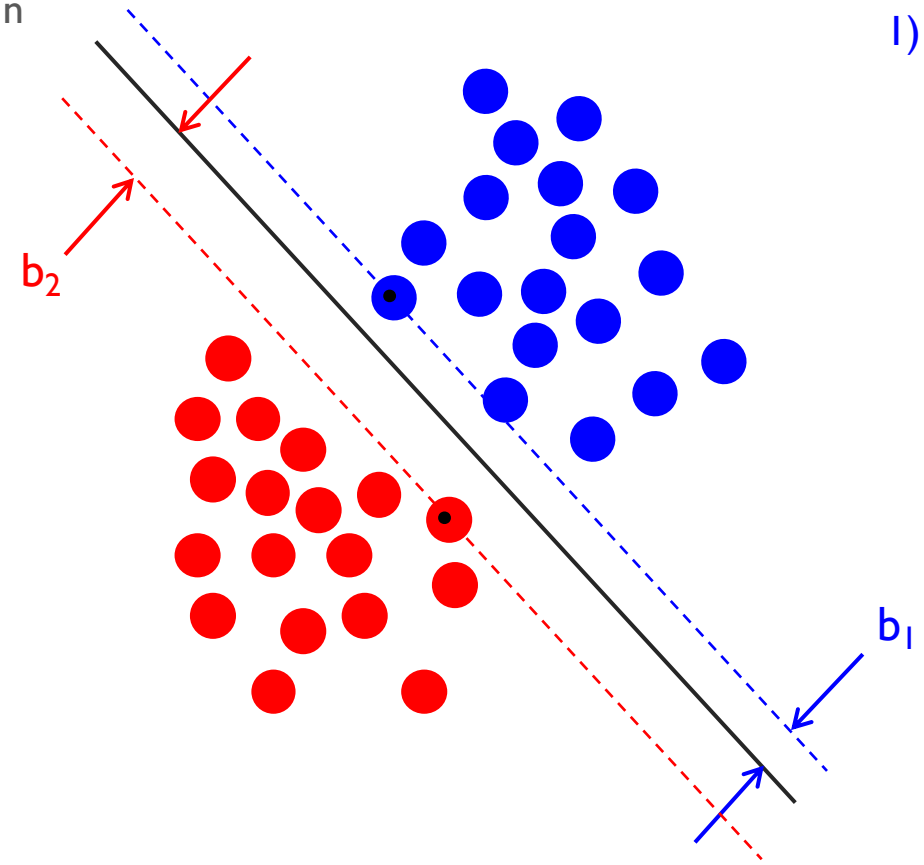


SVM: dos clases

Ideas clave de SVM:

I) $b_1 = b_2 = b$.

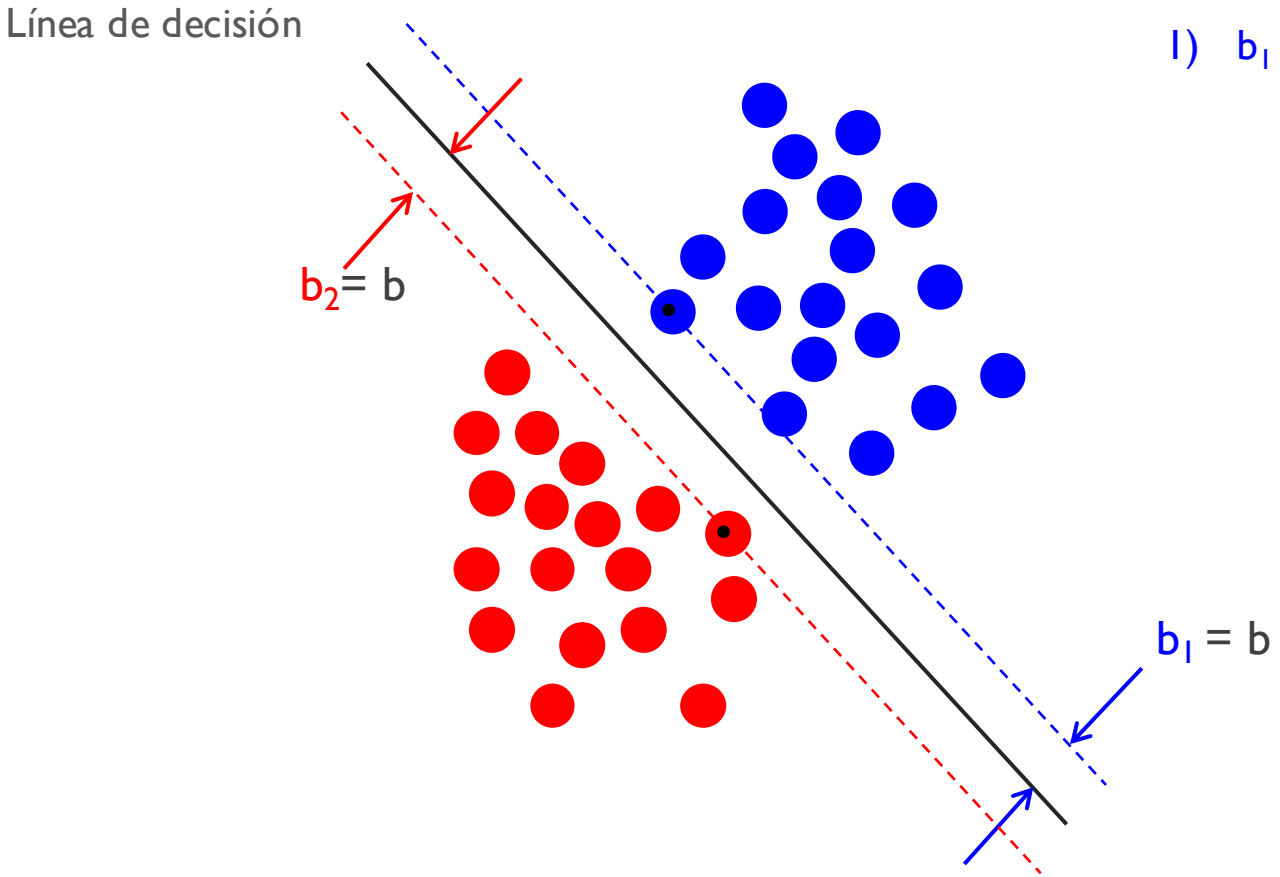
Línea de decisión



SVM: dos clases

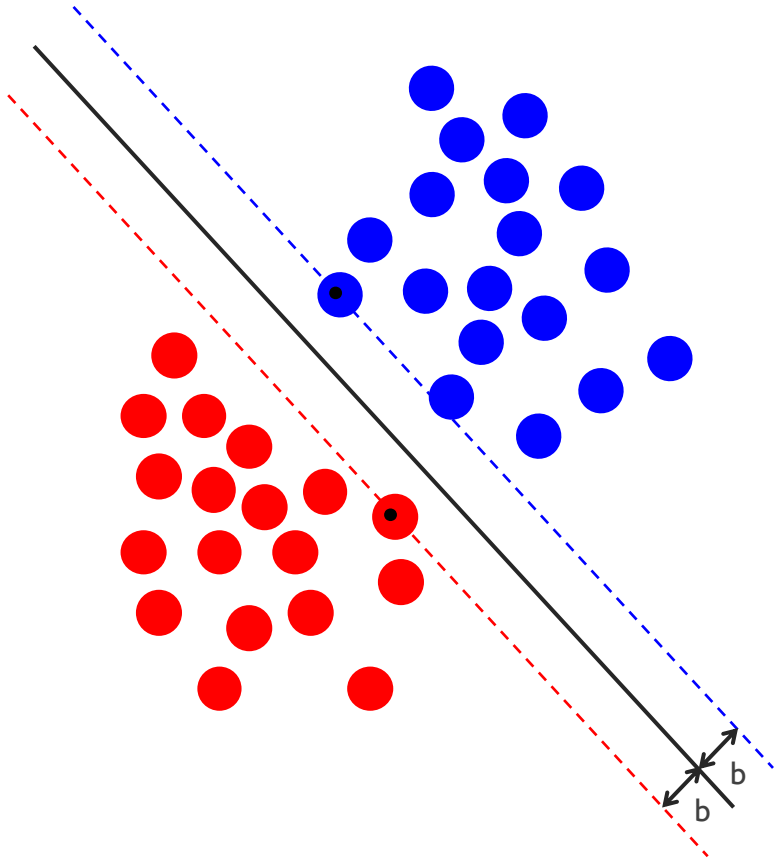
Ideas clave de SVM:

I) $b_1 = b_2 = b$.



SVM: dos clases

Línea de decisión

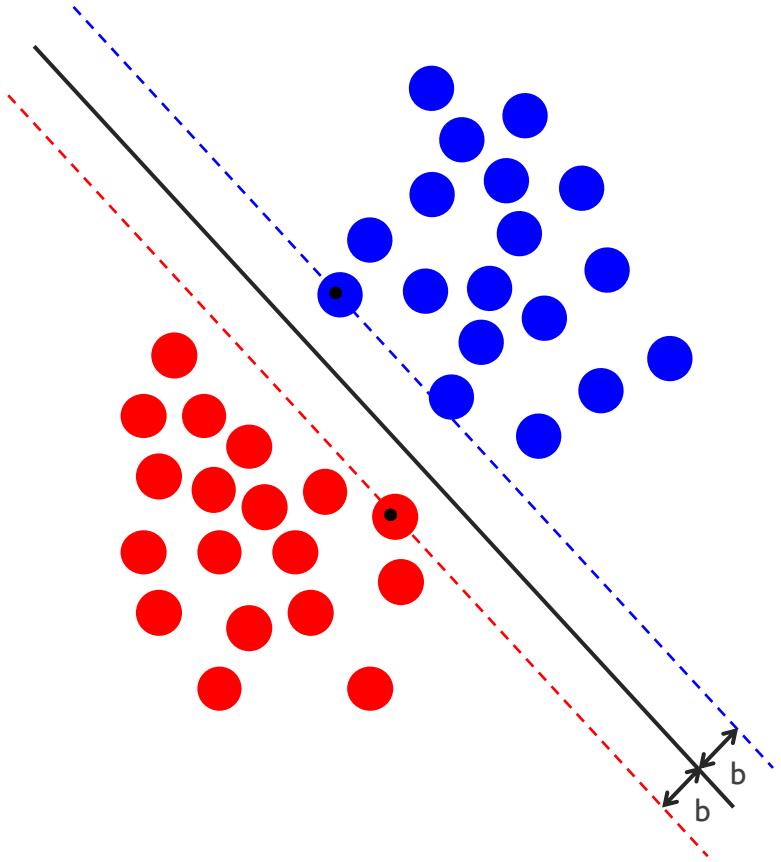


Ideas clave de SVM:

I) $b_1 = b_2 = b$.

SVM: dos clases

Línea de decisión



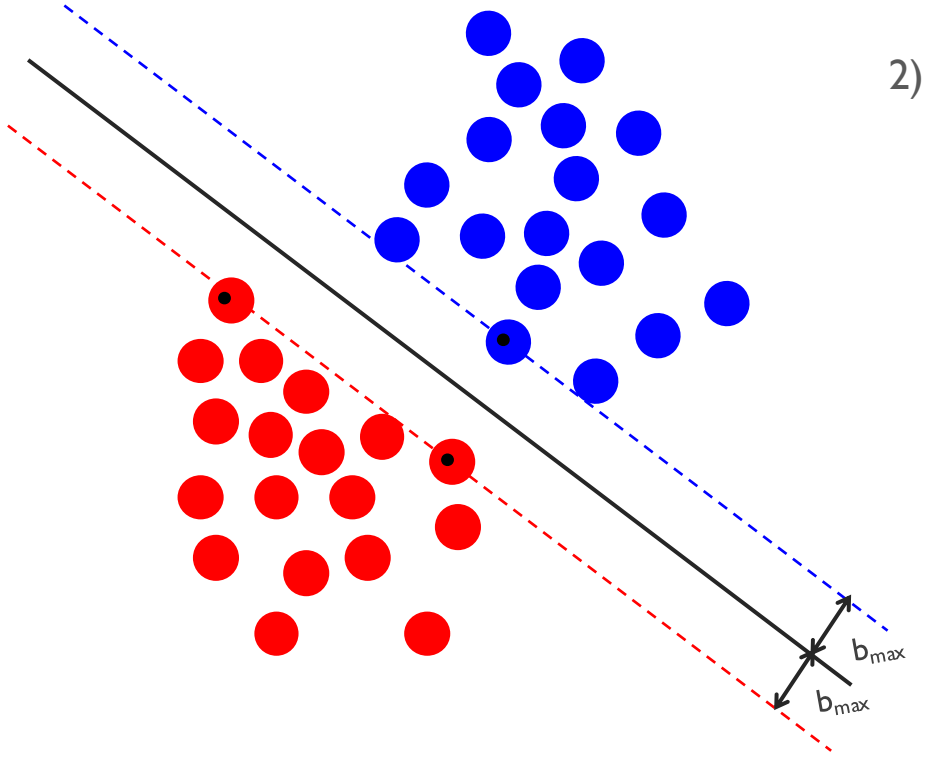
Ideas clave de SVM:

1) $b_1 = b_2 = b$.

2) b debe maximizarse.

SVM: dos clases

Línea de decisión

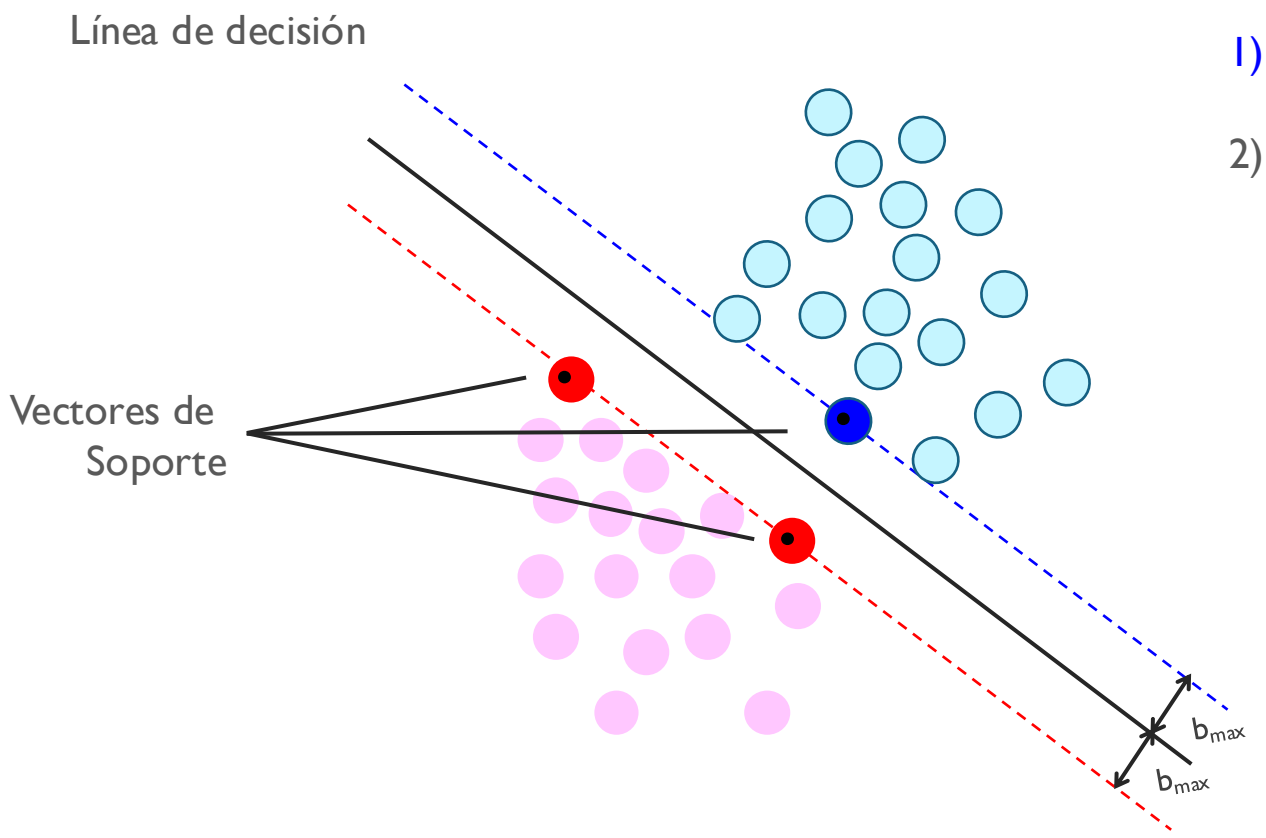


Ideas clave de SVM:

1) $b_1 = b_2 = b$.

2) b debe maximizarse.

SVM: dos clases

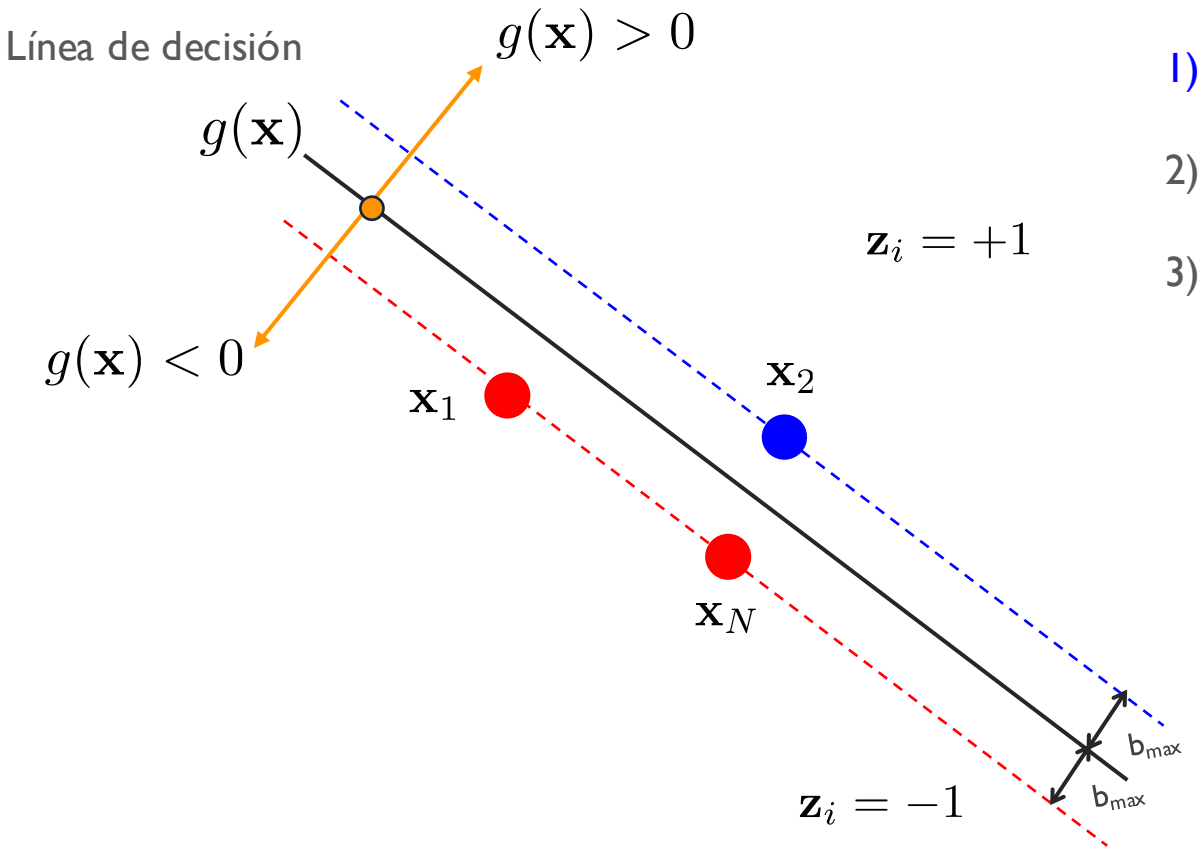


Ideas clave de SVM:

1) $b_1 = b_2 = b$.

2) b debe maximizarse.

SVM: dos clases



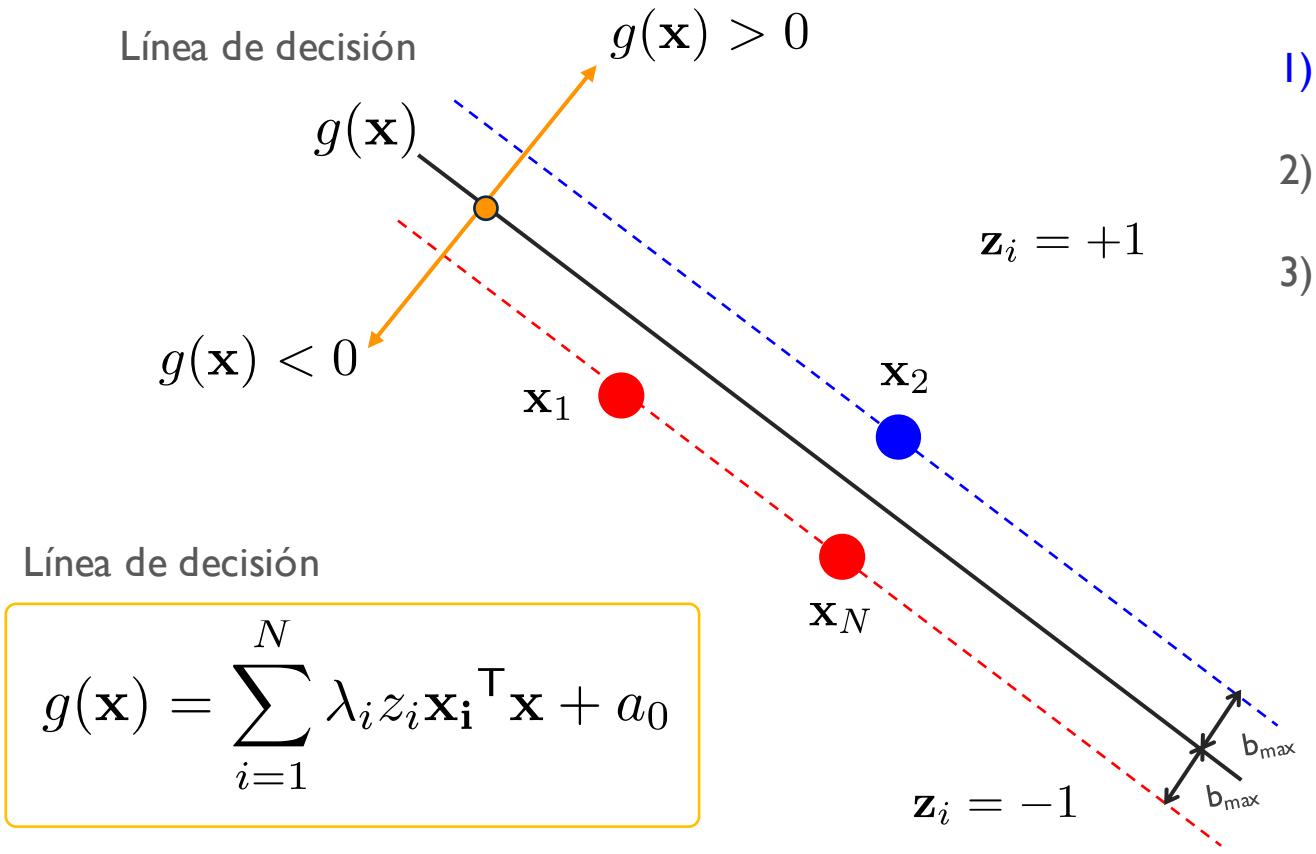
Ideas clave de SVM:

1) $b_1 = b_2 = b$.

2) b debe maximizarse.

3) Solución: $g(\mathbf{x})$

SVM: dos clases



Línea de decisión

$$g(\mathbf{x}) = \sum_{i=1}^{N} \lambda_i z_i \mathbf{x_i}^{\mathsf{T}} \mathbf{x} + a_0$$

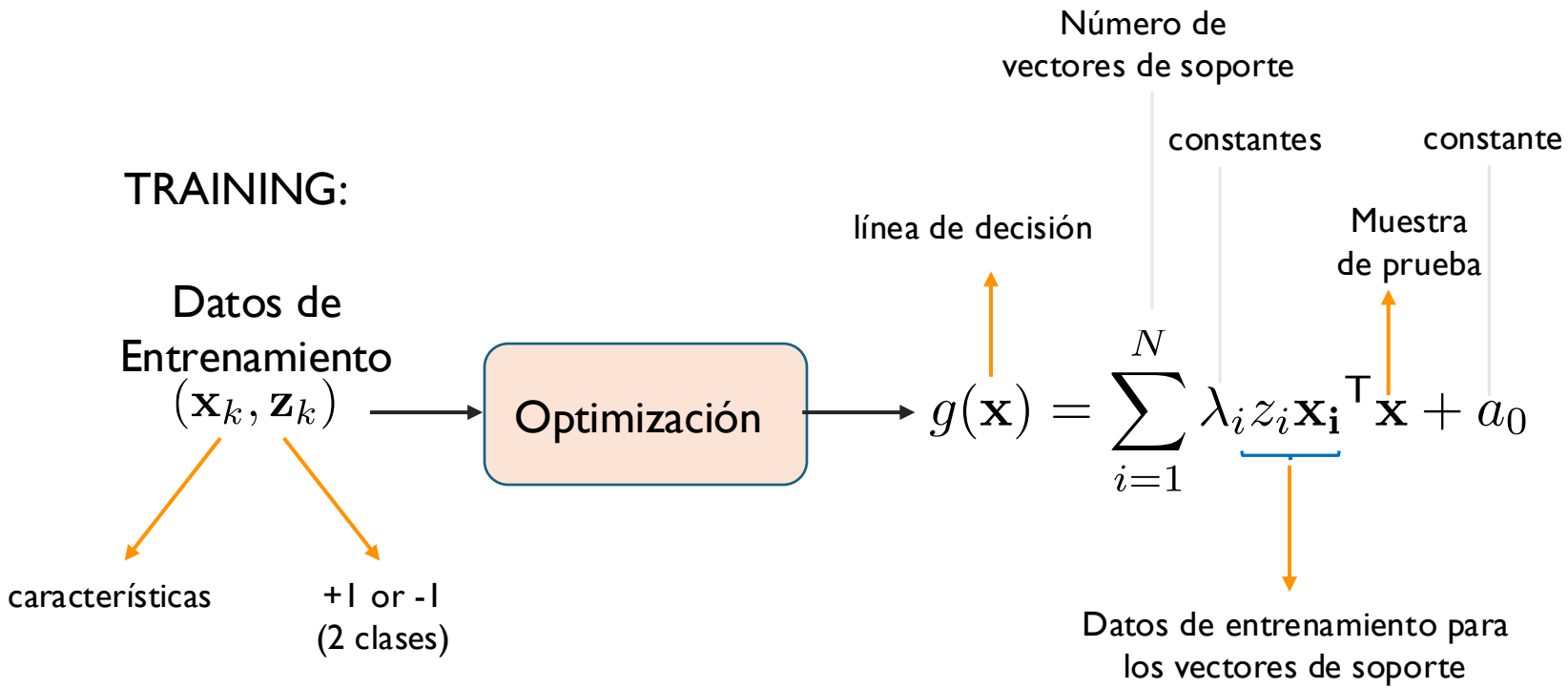
Ideas clave de SVM:

1) $b_1 = b_2 = b$.

2) b debe maximizarse.

3) Solución: $g(\mathbf{x})$

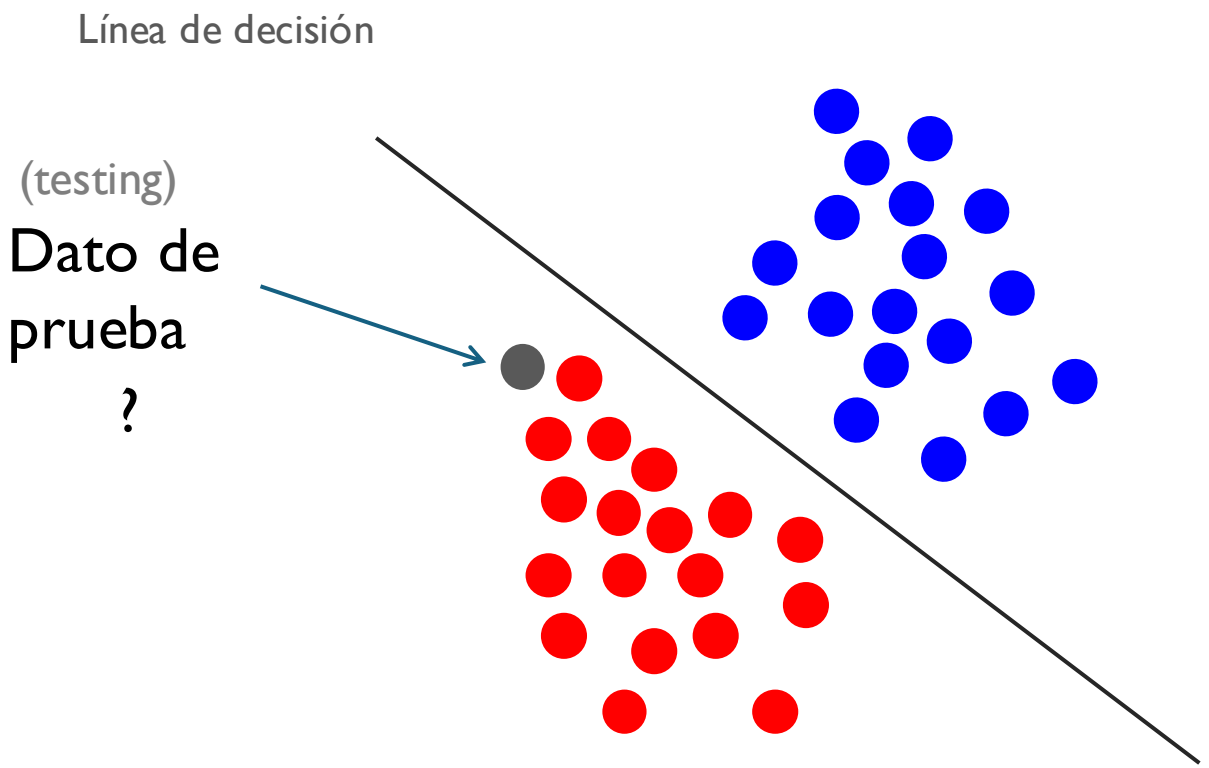
La solución de SVM



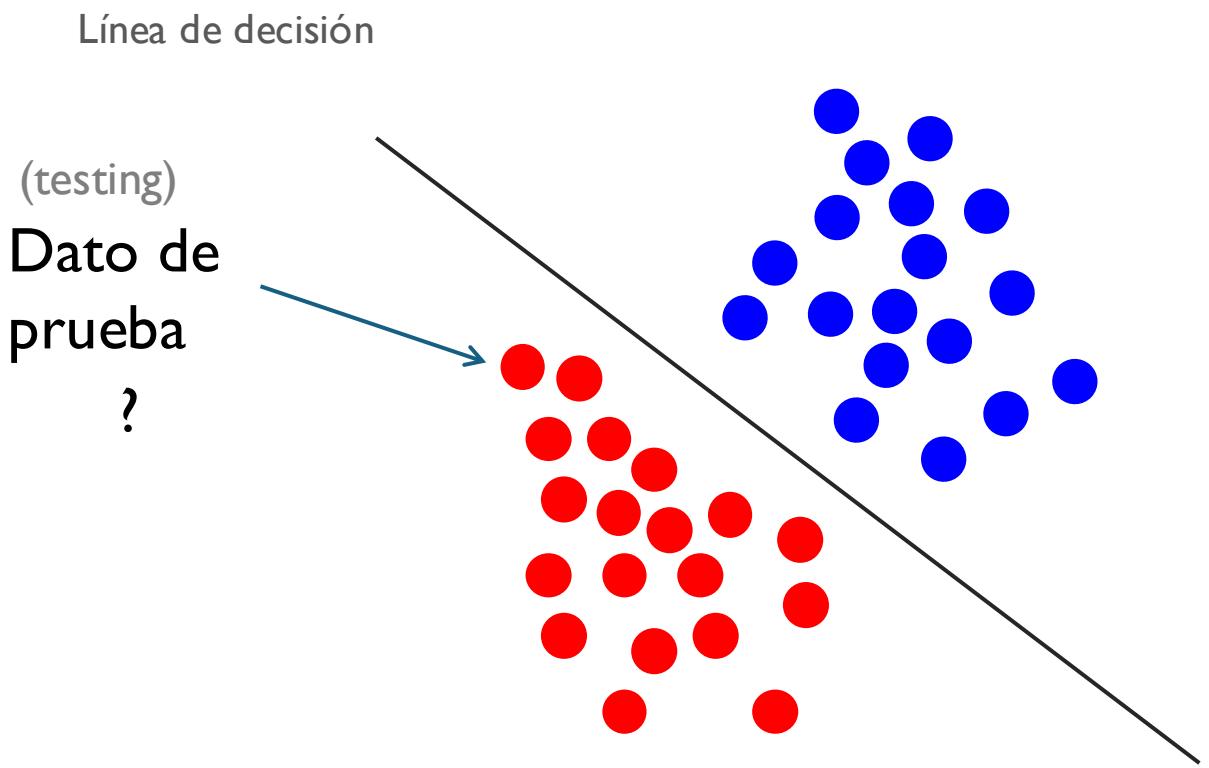
TESTING: si $g(\mathbf{x}) > 0$ entonces clase = +1

en caso contrario clase = -1

SVM: dos clases



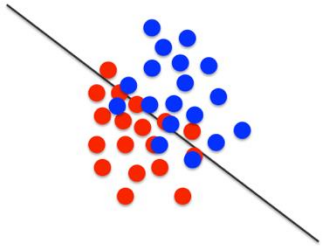
SVM: dos clases



SVM: Máquinas vectoriales de soporte

1) Lineal con separación perfecta

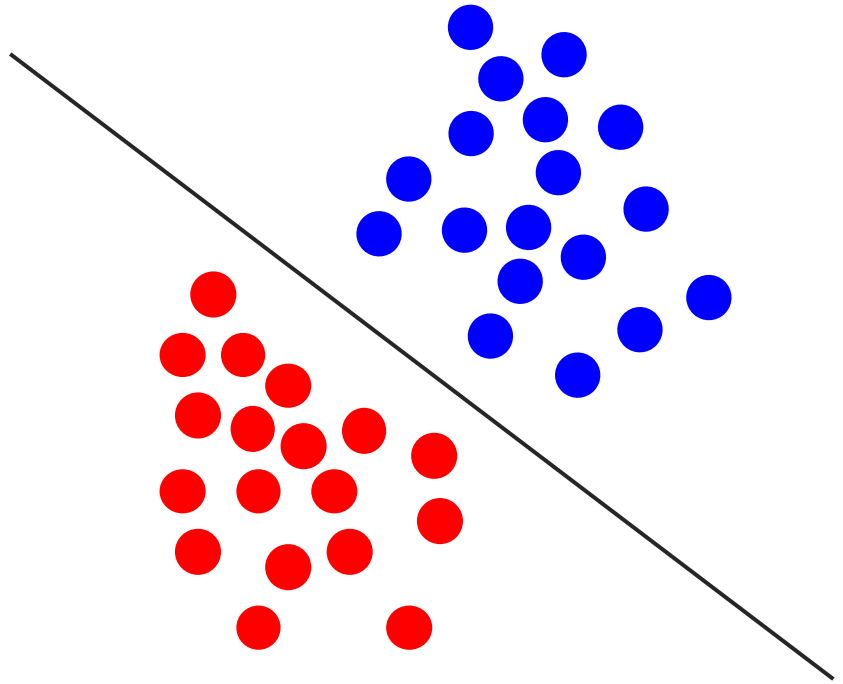
2) Lineal sin separación perfecta



3) No lineal

SVM: dos clases

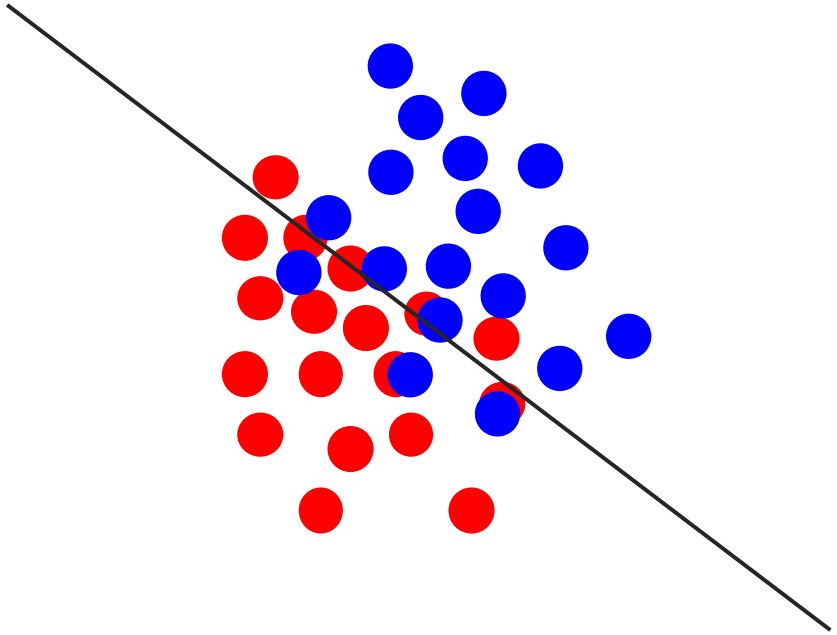
¿Cómo definir la línea de decisión cuando no hay una separación perfecta?



SVM: dos clases

¿Cómo definir la línea de decisión cuando no hay una separación perfecta?

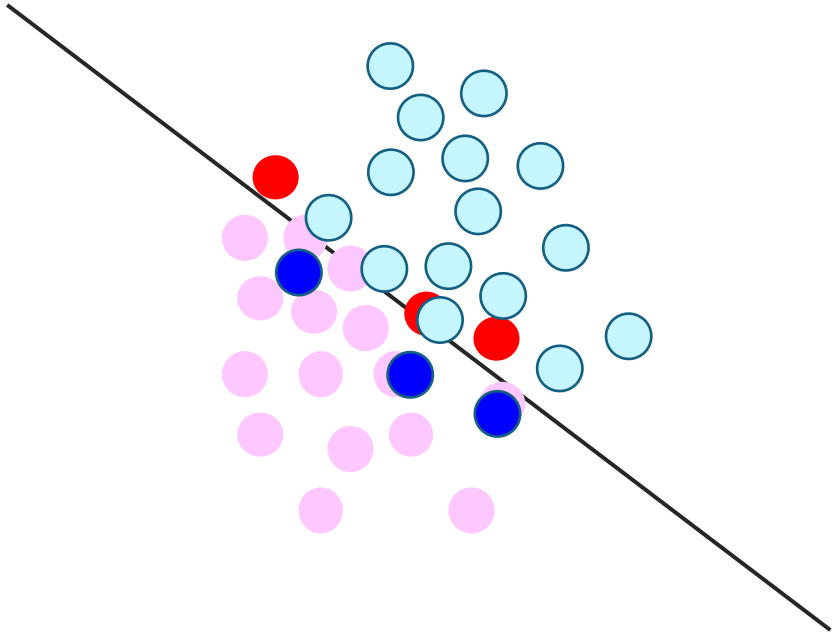
Consideramos sólo las muestras clasificadas erróneamente



SVM: dos clases

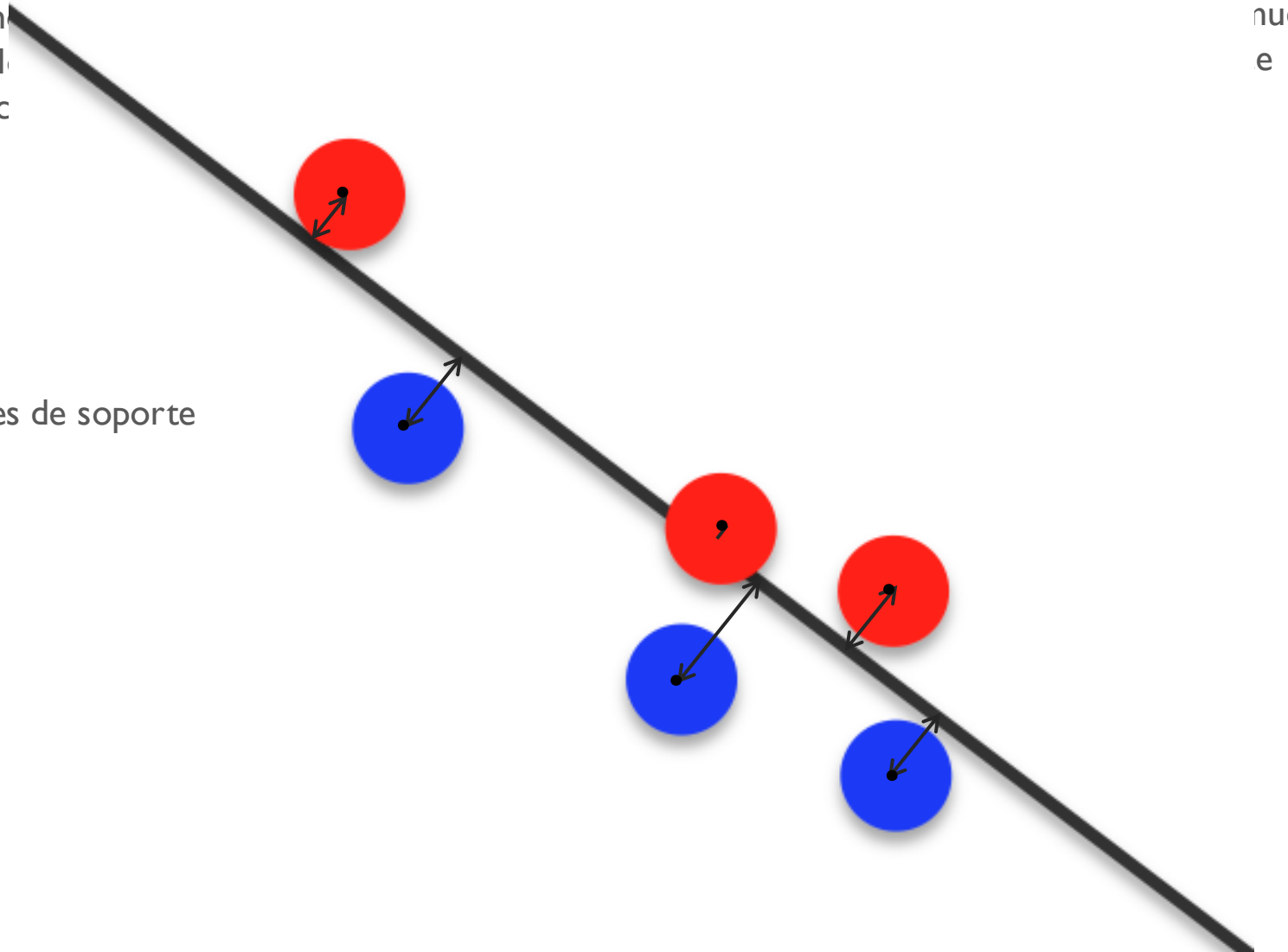
¿Cómo definir la línea de decisión cuando no hay una separación perfecta?

Consideramos sólo las muestras clasificadas erróneamente



SVM

¿Cóm nuestras
cuand e
perfec

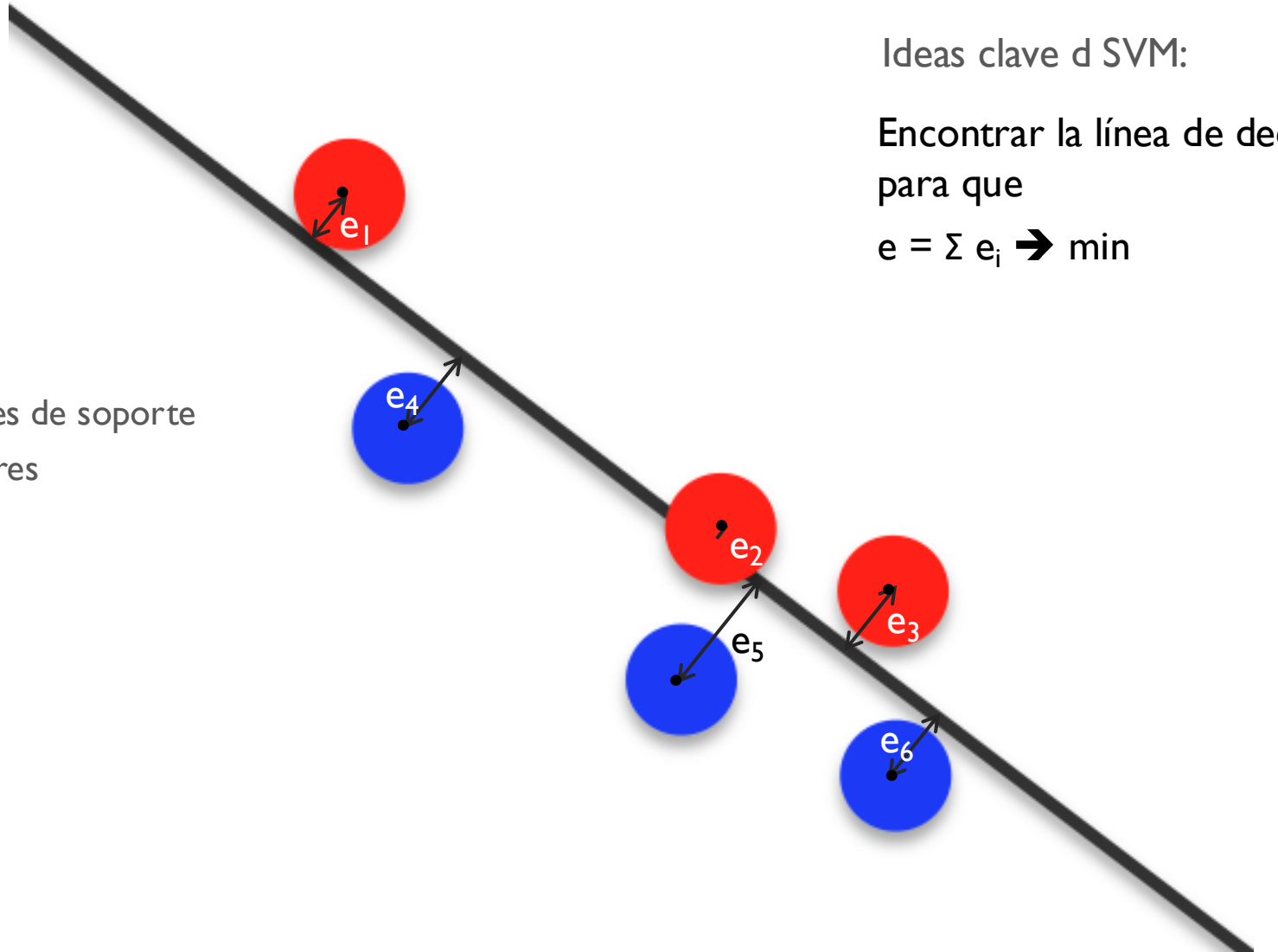


Vectores de soporte

Ideas clave d SVM:

Encontrar la línea de decisión para que

$e = \Sigma\ e_i$ ➔ min



Vectores de soporte

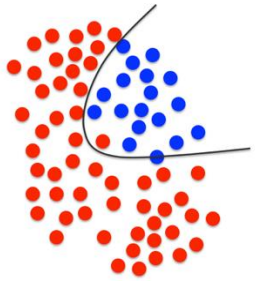
e_i: errores

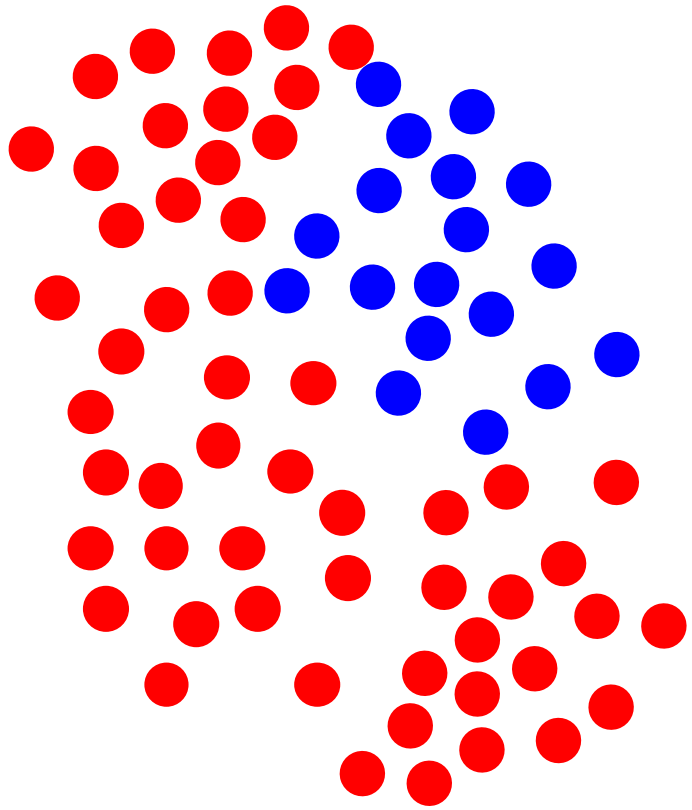
SVM: Máquinas vectoriales de soporte

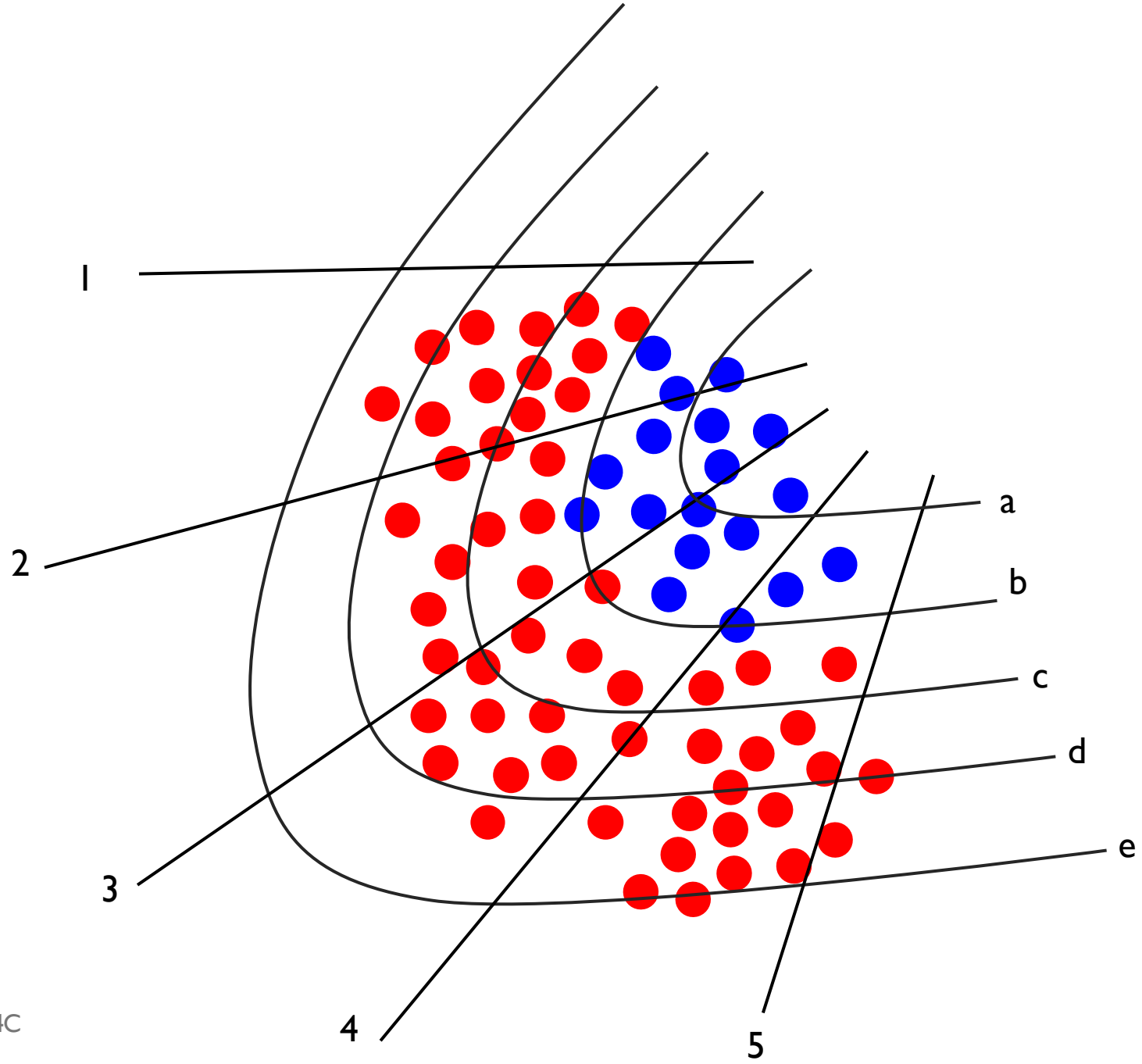
1) Lineal con separación perfecta

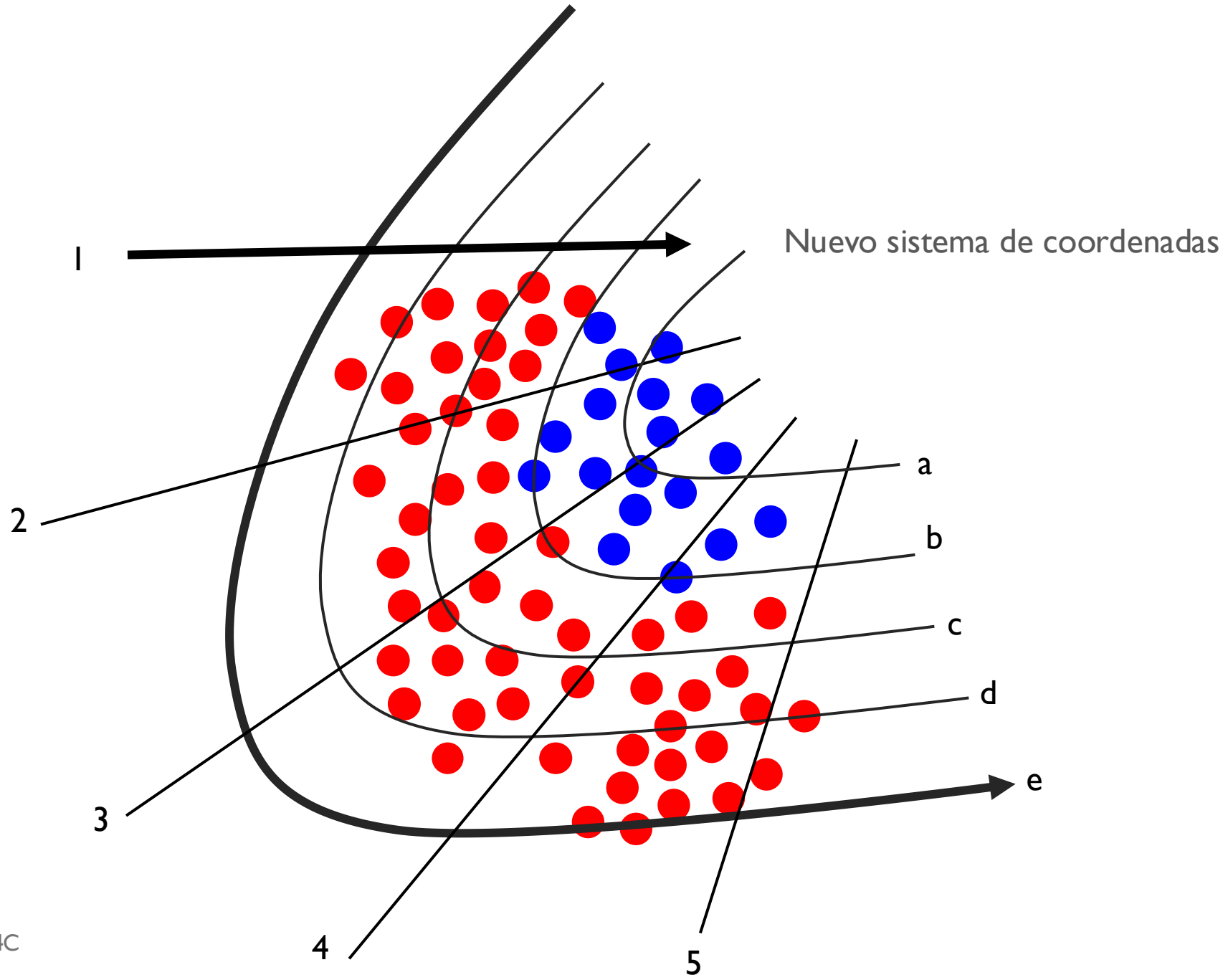
2) Lineal sin separación perfecta

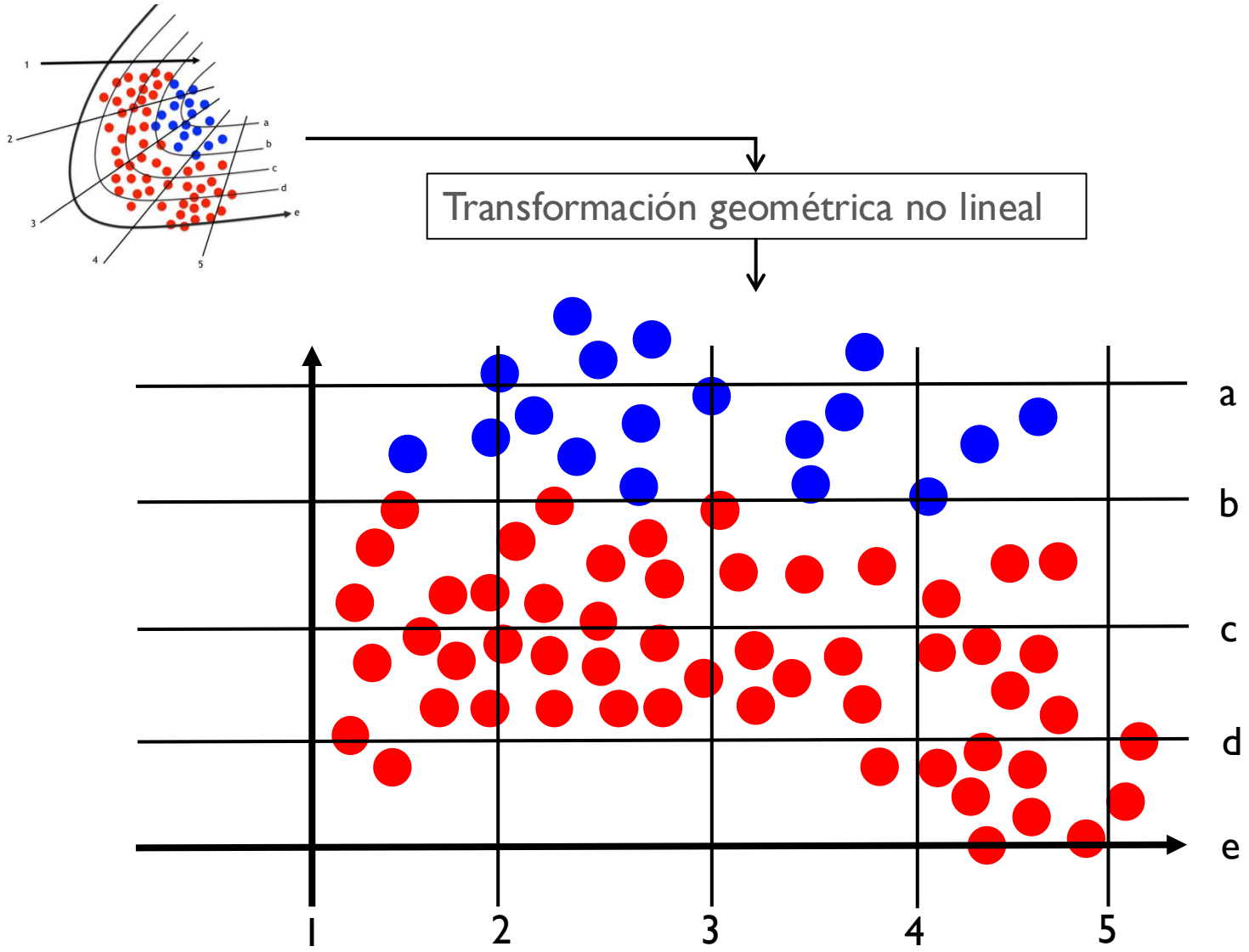
3) No lineal

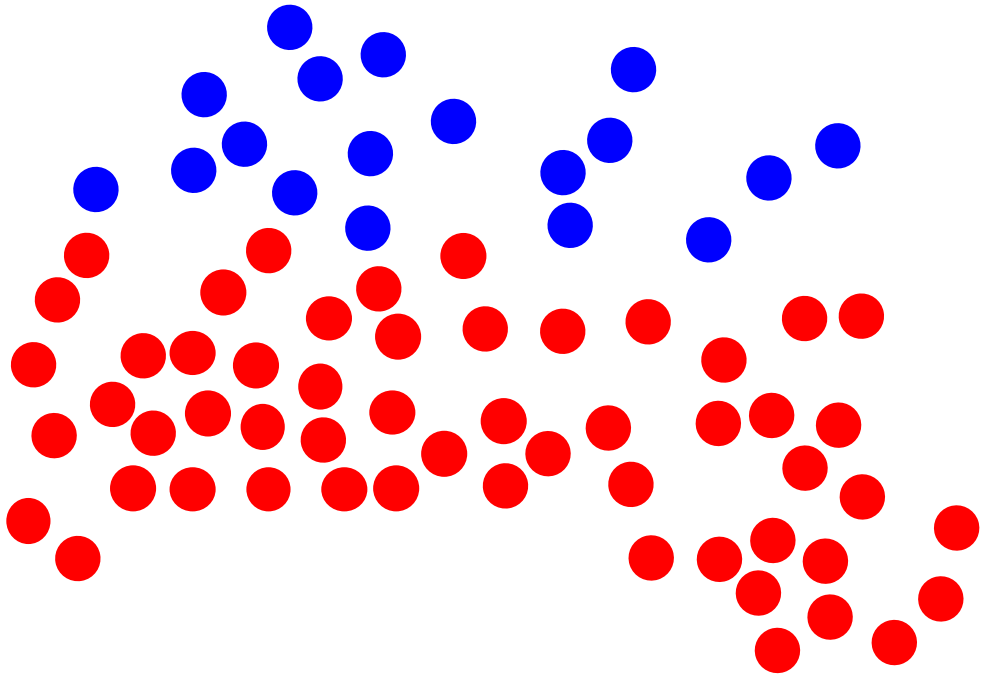




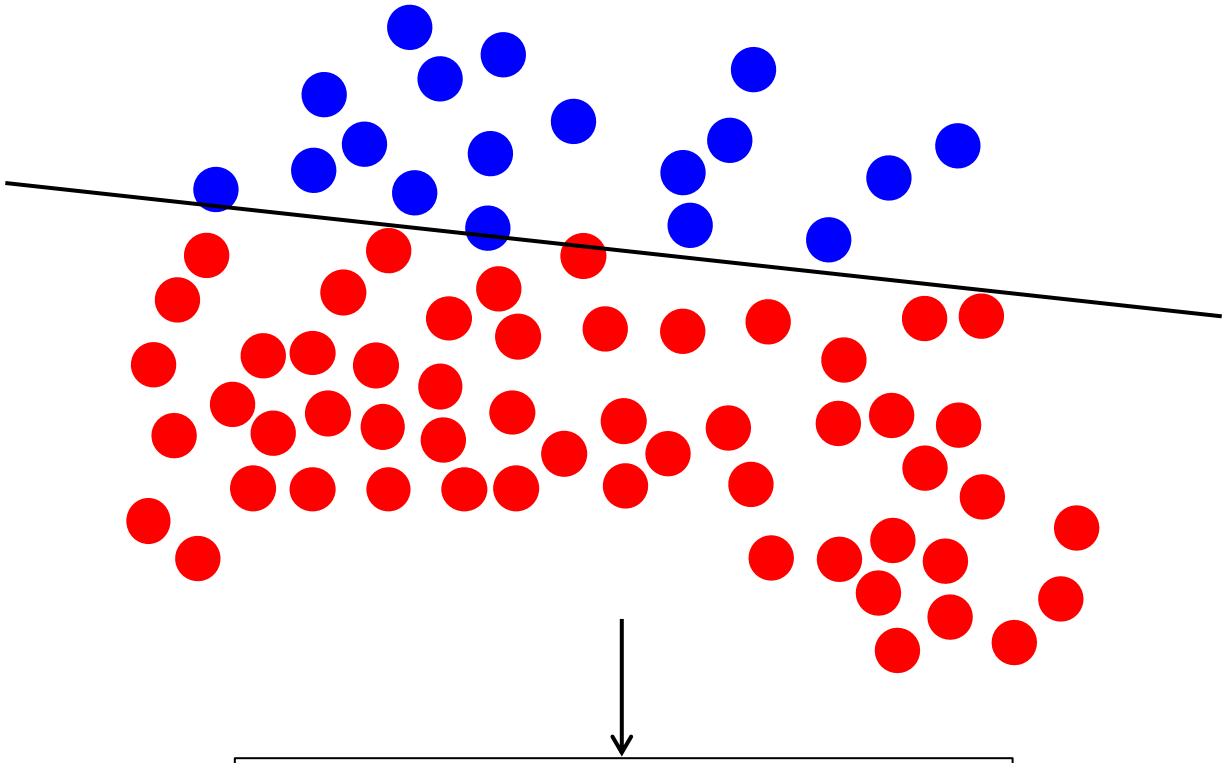

1
2
3
4
5
a
b
c
d
e


Nuevo sistema de coordenadas
1
2
3
4
5
a
b
c
d
e

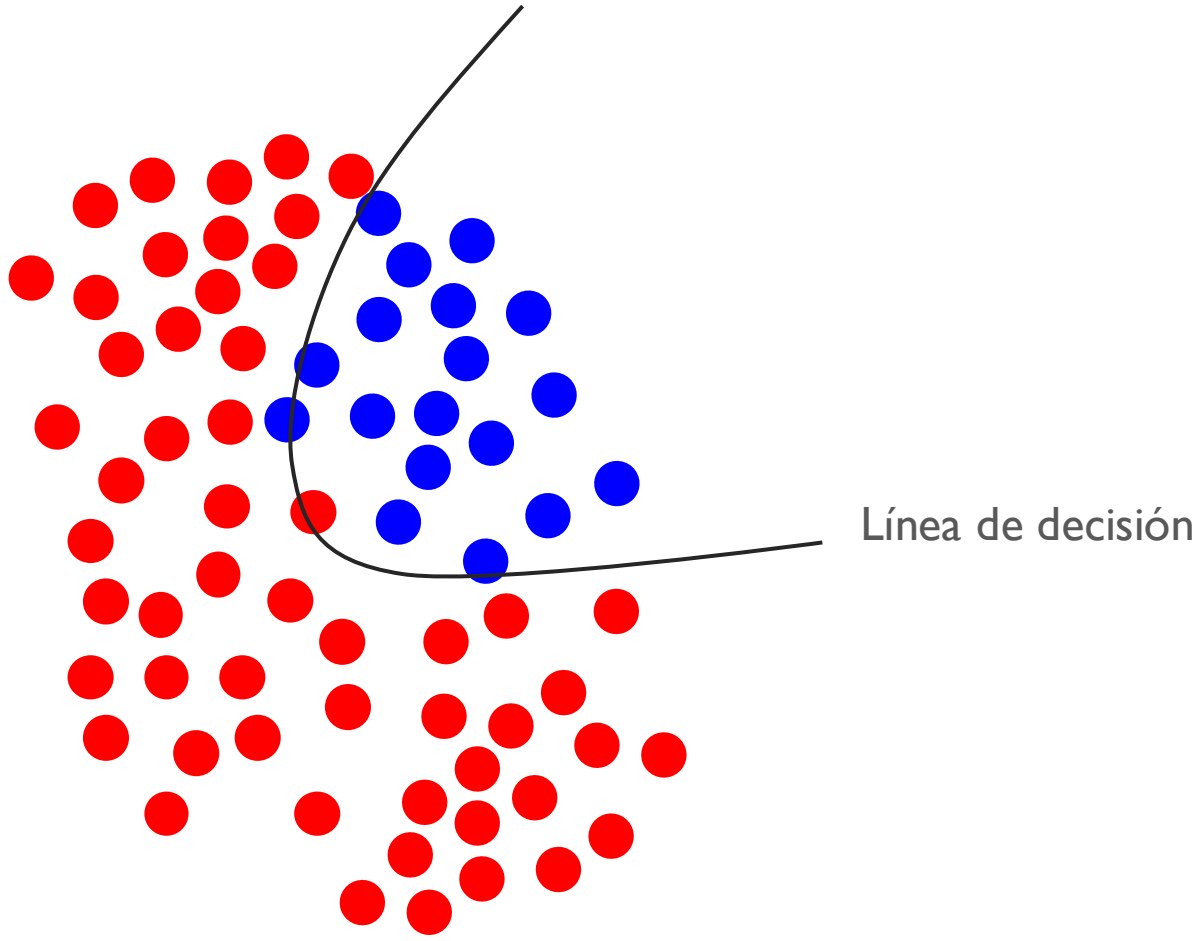

1
2
3
4
5
a
b
c
d
e
Transformación geométrica no lineal
a
b
c
d
e
1
2
3
4
5



SVM lineal en nuevo sistema de coordenadas

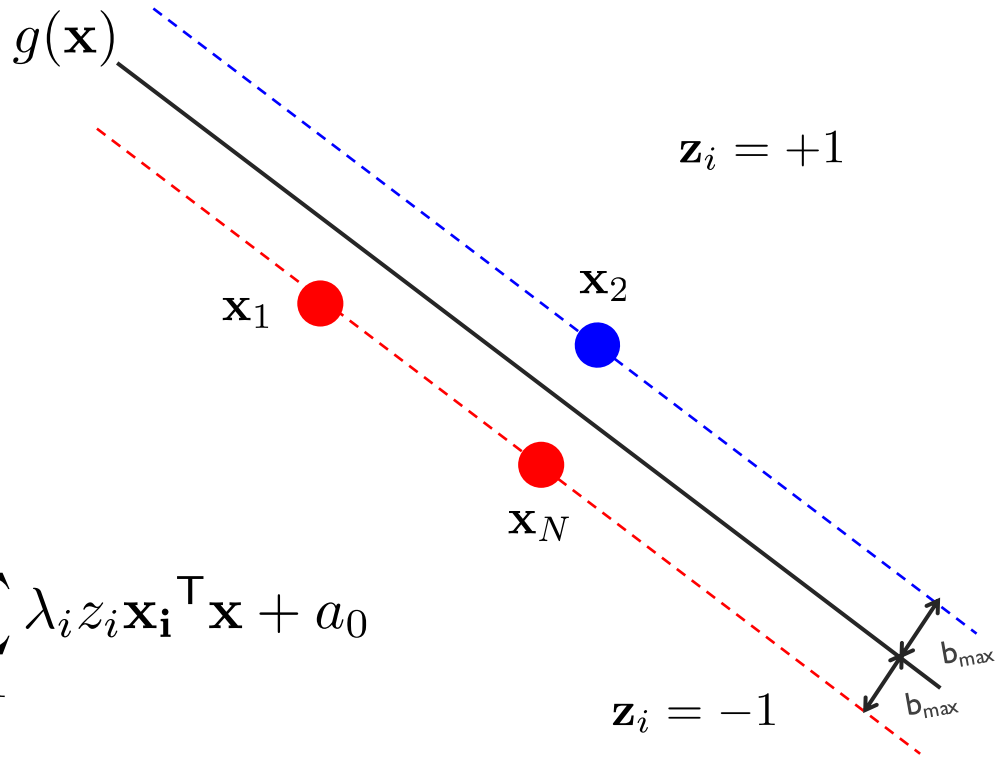


Transformación geométrica no lineal


Línea de decisión

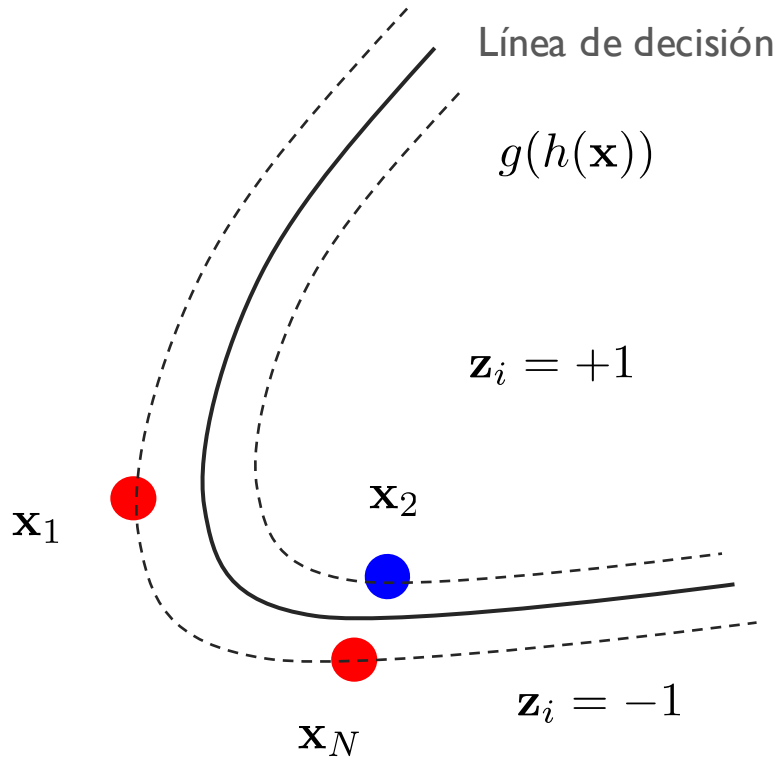
SVM: Máquinas vectoriales de soporte

Línea de decisión



$$g(\mathbf{x}) = \sum_{i=1}^{N} \lambda_i z_i \mathbf{x_i}^{\mathsf{T}} \mathbf{x} + a_0$$

SVM: Máquinas vectoriales de soporte



$$g(h(\mathbf{x})) = \sum_{i=1}^{N} \lambda_i z_i h(\mathbf{x_i})^{\mathsf{T}} h(\mathbf{x}) + a_0$$

$$g(h(\mathbf{x})) = \sum_{i=1}^{N} \lambda_i z_i < h(\mathbf{x_i}), h(\mathbf{x}) > + a_0$$

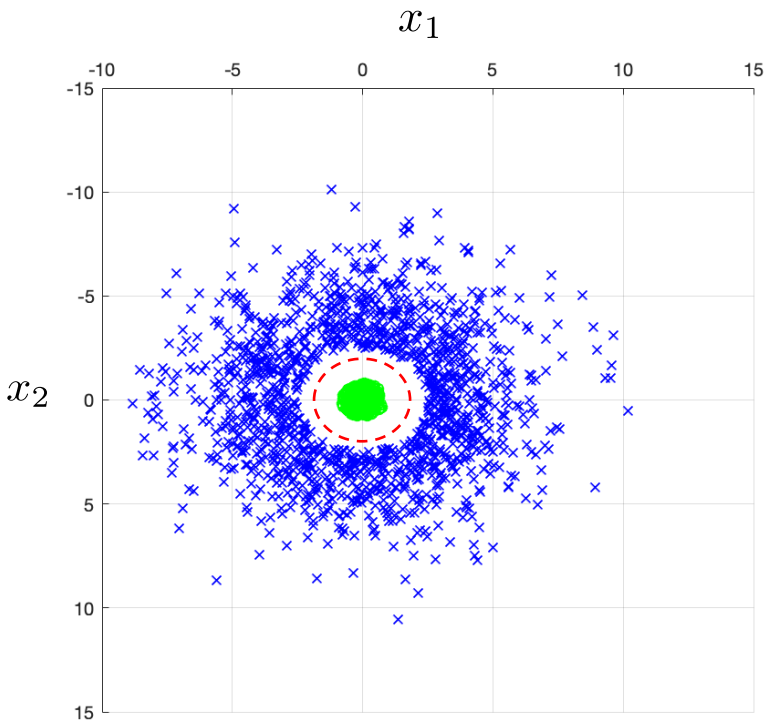
No se necesita $h(\mathbf{x})$, solo es necesario el kernel $< h(\mathbf{x_i}), h(\mathbf{x}) >$

SVM: Kernels

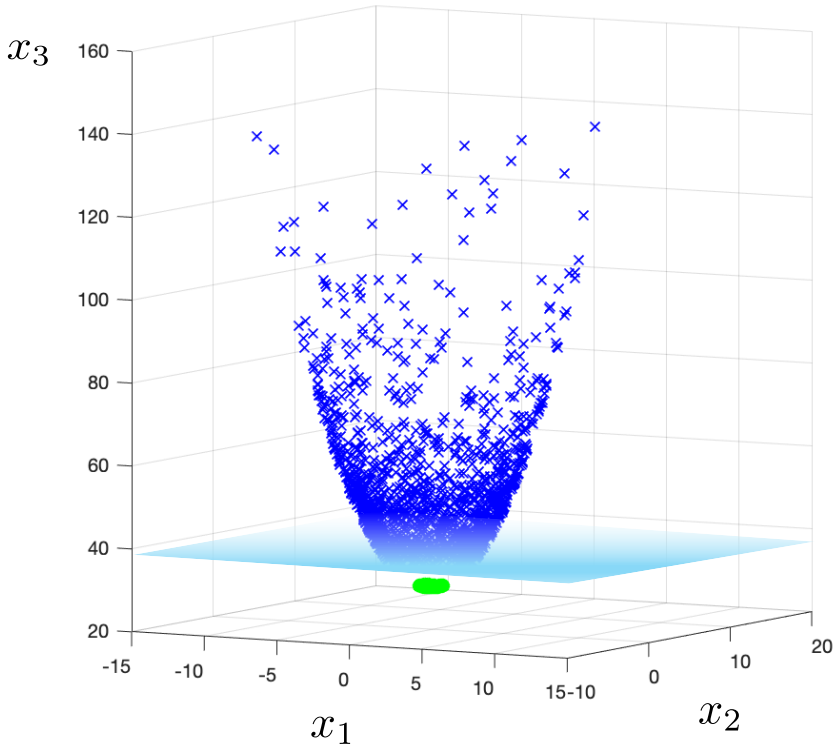
$$K(\mathbf{x}', \mathbf{x}) = \langle h(\mathbf{x}'), h(\mathbf{x}) \rangle =$$

linear	$\langle \mathbf{x}', \mathbf{x} \rangle$
polynomial	$(1 + \langle \mathbf{x}', \mathbf{x} \rangle)^n$
radial basis	$\exp(-\|\|\mathbf{x}' - \mathbf{x}\|\|^2/c)$
sigmoid	$\tanh(K_1 \langle \mathbf{x}', \mathbf{x} \rangle + K_2)$

SVM: El truco del Kernel







La separación lineal es imposible

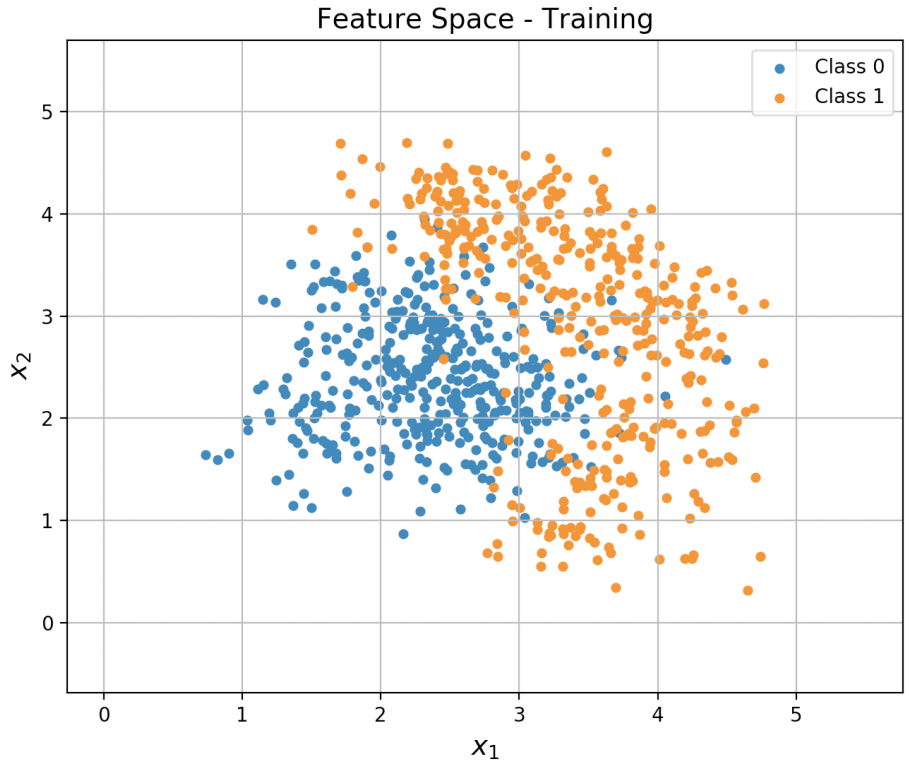
La separación lineal es perfecta

Ejemplos

Ejemplo

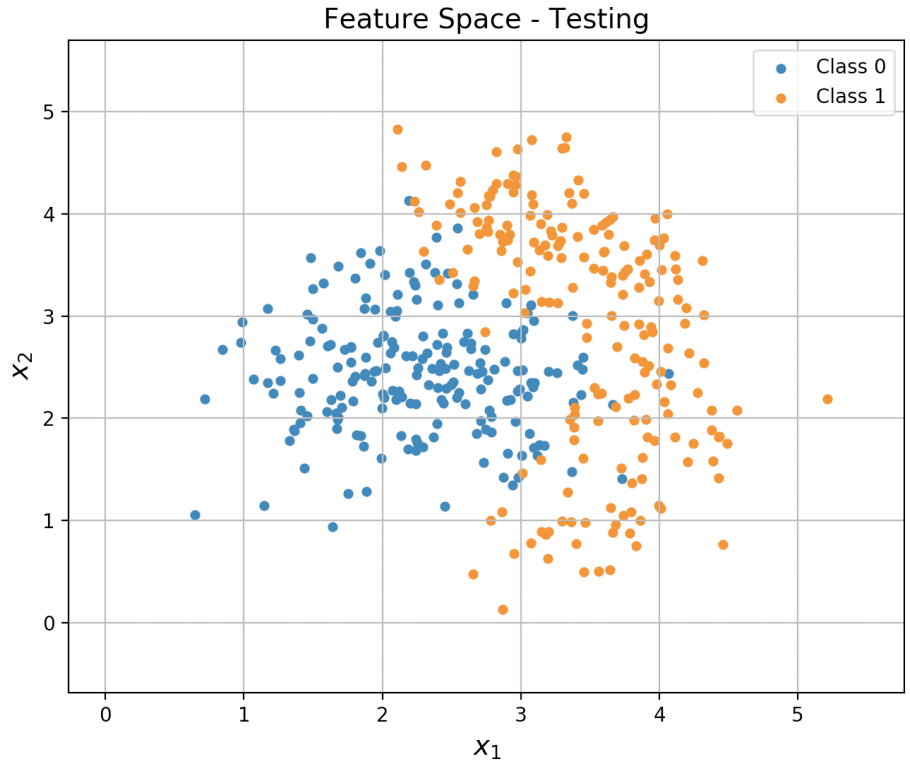
(X,d)

TRAINING



(X_t,d_t)

TESTING

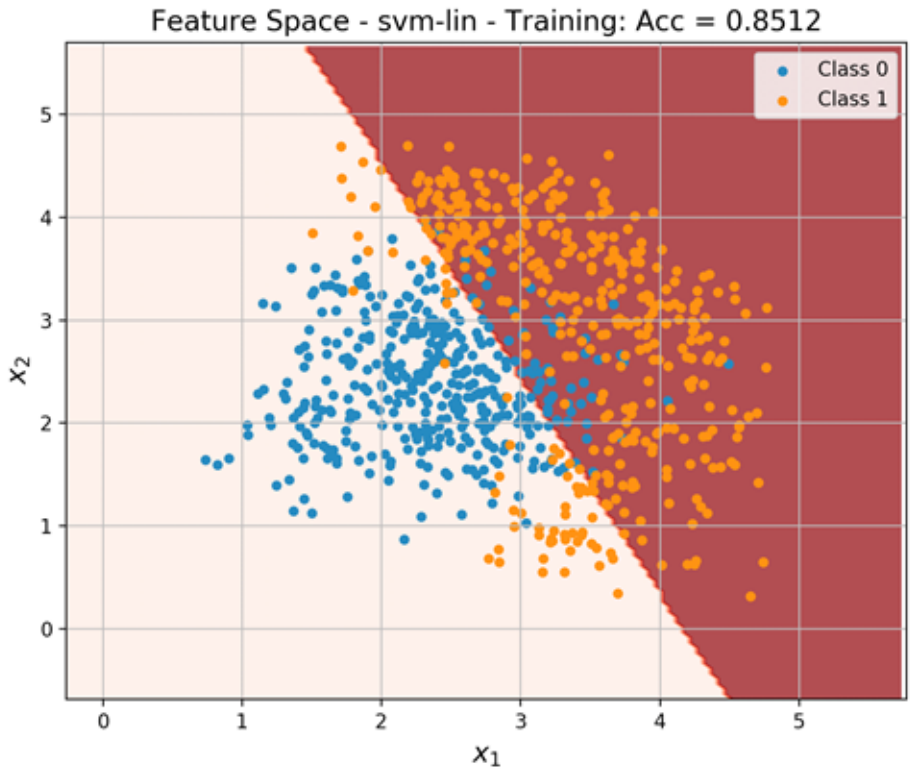


Ejemplo

SVM-LIN

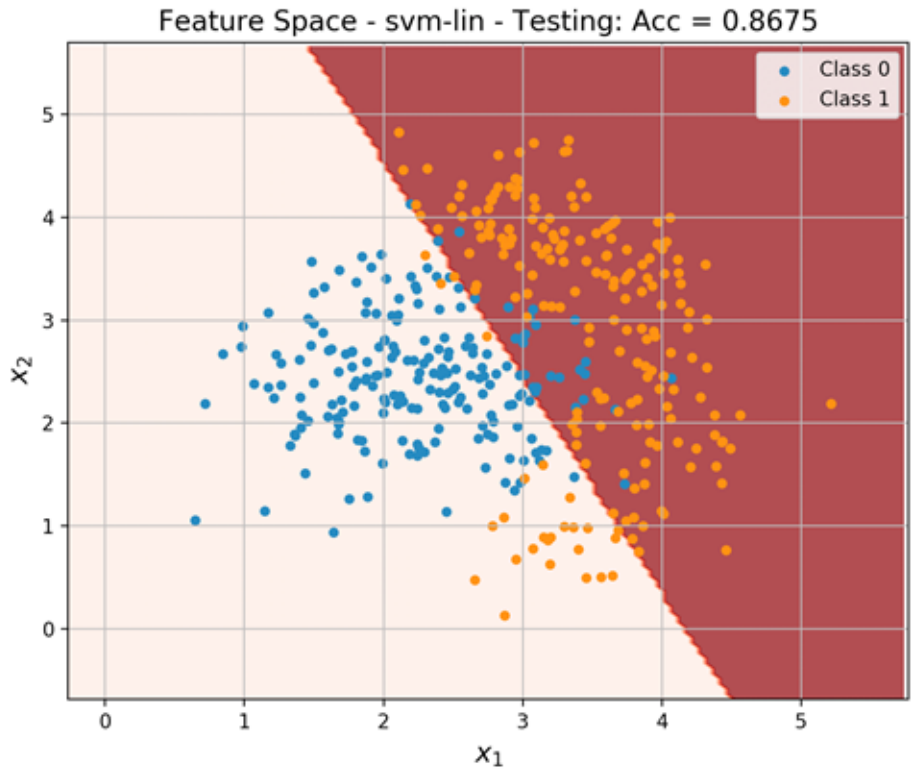
(X,d)

TRAINING



(X_t, d_t)

TESTING

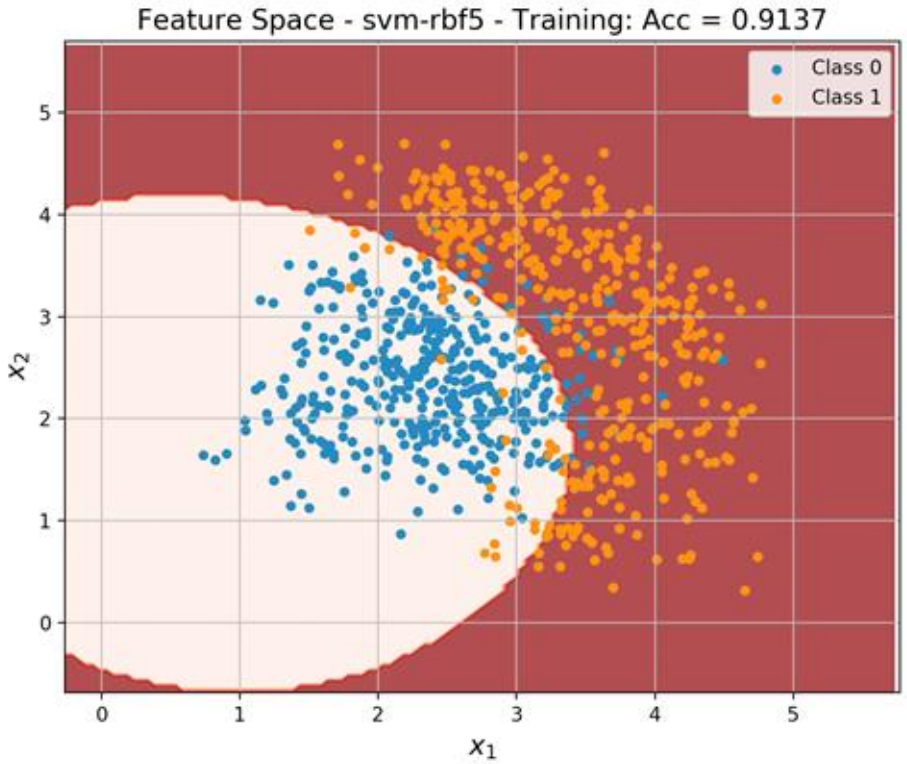


Ejemplo

SVM-RBF

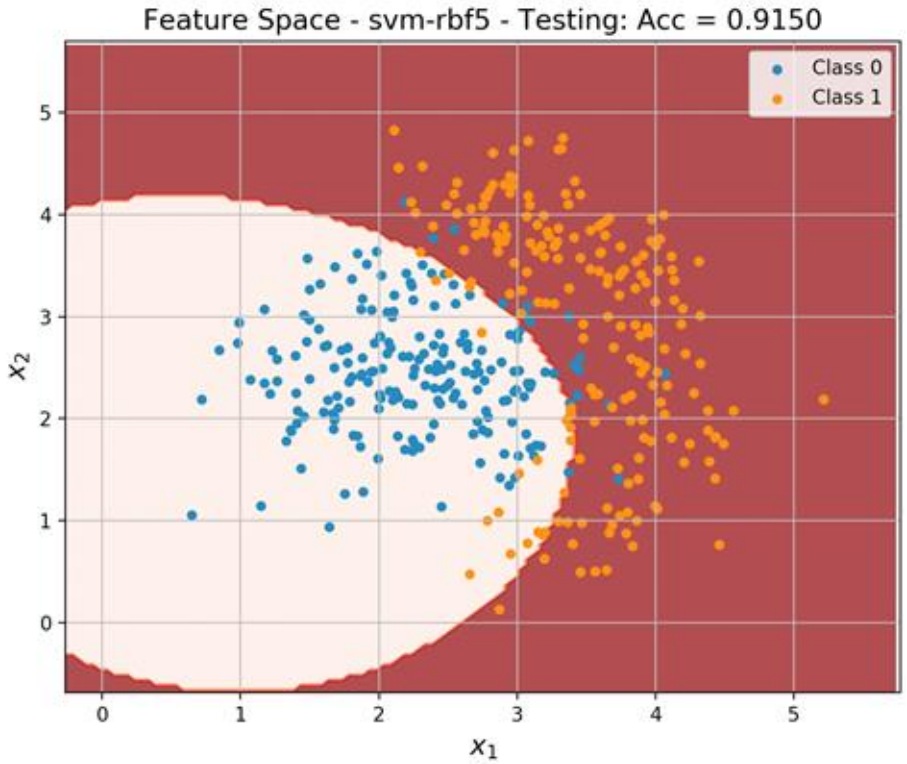
(X,d)

TRAINING



(X_t, d_t)

TESTING



SVM para N>2 clases

1. Uno contra el resto (OvR)

En OvR (one vs rest), se entrena un único clasificador binario para cada clase distinta.

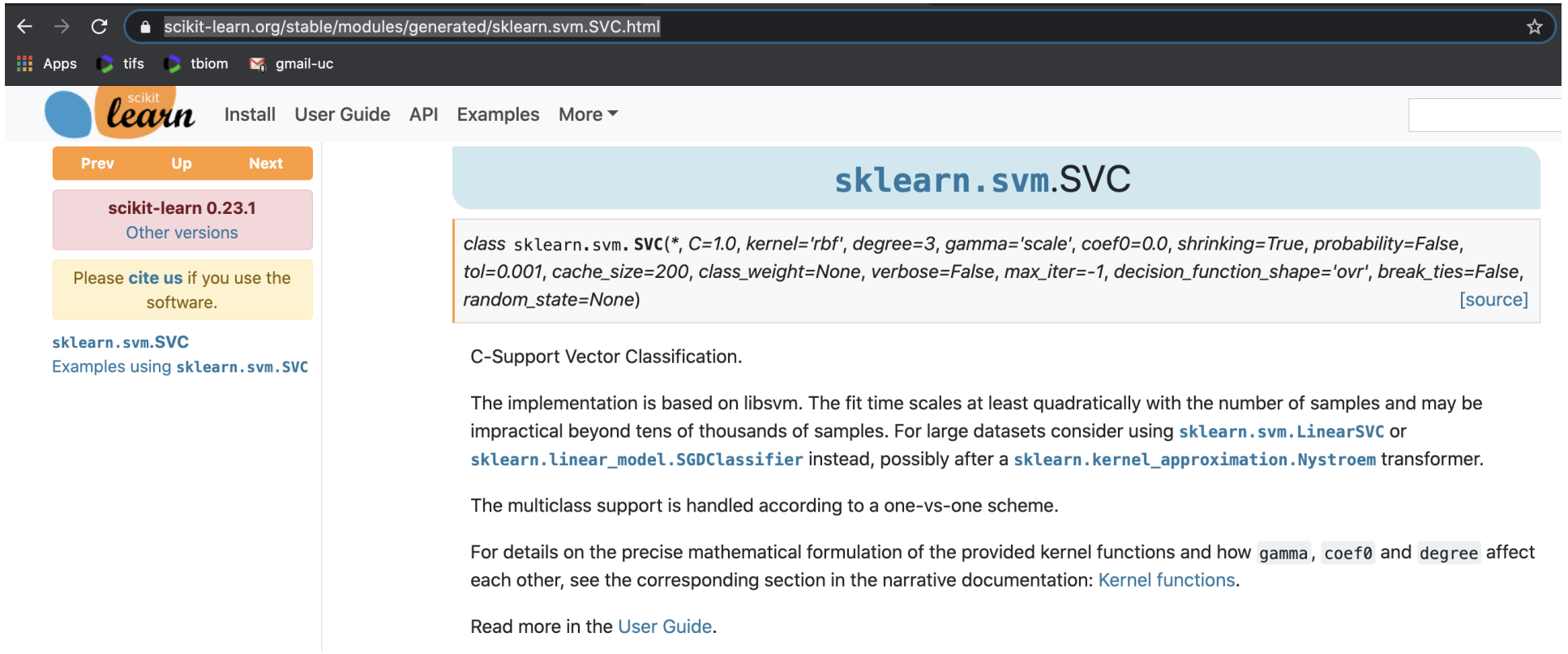
1. **Training:** Para las clases de N, se entrena N SVMs separados. Para cada clasificador, una clase se trata como la clase positiva (+1), mientras que todas las clases restantes se agrupan como la clase negativa (-1).
2. **Testing:** Un nuevo punto de datos se procesa a través de todos los N clasificadores. Cada modelo genera una puntuación de confianza (la distancia al hiperplano, recta, etc.). La predicción final se asigna a la clase cuyo clasificador da la puntuación más alta.
3. **Problema:** Los conjuntos de entrenamiento están muy desequilibrados. Por ejemplo, si tienes 10 clases iguales, cada clasificador se entrena con una proporción positiva a negativa de 1:9.

2. Uno contra Uno (OvO)

El enfoque OvO (one vs one) evita el desequilibrio del conjunto de datos entrenando un clasificador binario separado para cada posible par de clases.

1. **Training:** Para las N clases, se requiere entrenar N(N - 1)/2 clasificadores. Por ejemplo, con 4 clases (A, B, C, D), entrenas 6 clasificadores: (A vs B), (A vs C), (A vs D), (B vs C), (B vs D) y (C vs D). Cada clasificador se entrena sólo con los datos que pertenecen a esas dos clases específicas.
2. **Testing:** Al clasificar una nueva muestra de testing, se prueba en todos los N(N - 1)/2 clasificadores entrenados. Cada clasificador emite un "voto" por la clase ganadora. La predicción final se determina por mayoría de votos.
3. **Problema:** El número de clasificadores necesarios crece de forma cuadrática. Si hay 100 clases, es necesario entrenar 4.950 modelos!

SVM in SKLEARN - Python



SVM en SKLEARN - Python

Intuitivamente, el parámetro gamma define hasta dónde llega la influencia de un único ejemplo de entrenamiento, con valores bajos que significan "lejos" y valores altos que significan "cerca". Los parámetros gamma pueden verse como la inversa del radio de influencia de las muestras seleccionadas por el modelo como vectores de soporte.

El parámetro C compensa la clasificación correcta de los ejemplos de entrenamiento con la maximización del margen de la función de decisión. Para valores mayores de C, se aceptará un margen menor si la función de decisión clasifica correctamente todos los puntos de entrenamiento. Un valor menor de C fomentará un margen mayor y, por tanto, una función de decisión más simple, a costa de la precisión del entrenamiento. En otras palabras, ``C`` se comporta como un parámetro de regularización en la SVM.

https://scikit-learn.org/stable/auto_examples/svm/plot_rbf_parameters.html

Hemos aprendido:

- Qué es el margen
- Por qué una buena clasificación maximiza el margen
- Cuáles son las ideas clave de un clasificador SVM
- Cómo se realizar un clasificador SVM lineal
- Qué es un Kernel
- Cómo se realiza un clasificador SVM no lineal